pkg/vm + Image-Cache Bundled Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended, per Group B pattern) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Close §6.K-debt criterion (2) by adding `submodules/containers/pkg/vm/` (QEMU full-system VM orchestration mirroring `pkg/emulator/AndroidMatrixRunner`'s shape) + a generic `pkg/cache/` (CAS image cache used by both `pkg/vm/` and the refactored `pkg/emulator/`); bundle Group C's image-cache-management item; ship 2 Lava-side consumers (cross-arch signing × 9 configs and cross-OS distro × 3) that produce attestations matching Group B's row schema so `scripts/tag.sh`'s 3 Group B gates work unchanged.

Architecture: 3 new Containers packages (`pkg/cache/`, `pkg/vm/`, `cmd/vm-matrix/`) + an internal refactor of `pkg/emulator/` to route image fetch through `pkg/cache/`. Lava-domain pieces stay Lava-side per Decoupled Reusable Architecture: `tools/lava-containers/vm-images.json` (project-side manifest with the 9 qcow2 SHA-256 entries) + 2 thin-glue scripts wrapping `cmd/vm-matrix` for the consumer matrices. Every commit's body carries a Bluff-Audit stamp; pre-push Check 4 (gate-shaping file ⇒ stamp required) enforces.

Tech Stack: Go 1.25 (`encoding/json`, `os/exec`, `crypto/sha256`, `sync`, `golang.org/x/crypto/ssh`, `syscall.Flock`, stdlib `testing`); QEMU 6.0+ (qemu-system-x86_64, qemu-system-aarch64, qemu-system-riscv64); bash 5+ + jq; existing pre-push hook from Group A-prime + B; `pkg/emulator/cleanup.go::KillByPort` (already strict-adjacent token matcher; reusable).

File Structure

File	Responsibility
`submodules/containers/pkg/cache/manifest.go`	`Manifest`, `ImageEntry`, `LoadMa` malformed SHA-256, unkn
`submodules/containers/pkg/cache/store.go`	

File	Responsibility
	`Store` interface; `FilesystemS` `containers-images/blobs/sha` rejection-on-mismatch
`submodules/containers/pkg/cache/manifest_test.go`	Schema validation tests +
`submodules/containers/pkg/cache/store_test.go`	Get/Verify/Refresh tests ag test; SHA-mismatch rejecti
`submodules/containers/pkg/vm/types.go`	`VMTarget`, `BootResult`, `VMConf` (Gating + per-row Diag/Fai
`submodules/containers/pkg/vm/qemu.go`	`QEMUVM` impl of `VM` interface; `golang.org/x/crypto/ssh`; SC shutdown
`submodules/containers/pkg/vm/teardown.go`	QEMUVM.Teardown (30s S `emulator/cleanup.KillByPort`
`submodules/containers/pkg/vm/matrix.go`	`QEMUMatrixRunner` mirroring `parseScriptFailures` + work
`submodules/containers/pkg/vm/qemu_test.go`	Fake SSH server + fake QE tests; falsifiability rehearsa
`submodules/containers/pkg/vm/teardown_test.go`	Fast-path skip-on-mismatch `TestTeardown_FastPath_Skips`
`submodules/containers/pkg/vm/matrix_test.go`	`stubVM` driving QEMUMatri FailureSummaries-from-scr
`submodules/containers/cmd/vm-matrix/main.go`	Thin CLI: flag parser; const `emulator-matrix`
`submodules/containers/pkg/emulator/types.go`	(modify) extend `MatrixConfi`
`submodules/containers/pkg/emulator/android.go`	(modify) route system-imag under `ANDROID_SDK_ROOT` ANI **unchanged**
`submodules/containers/pkg/emulator/android_test.go`	(modify) add `TestAndroidEmulator_Boot_Fe` — falsifiability rehearsal
`Lava/tools/lava-containers/vm-images.json`	Project-side manifest with + 1 Android system-image
`Lava/scripts/run-vm-signing-matrix.sh`	Thin glue invoking `cmd/vm-m` per-row `signing_match` from
`Lava/scripts/run-vm-distro-matrix.sh`	Thin glue invoking `cmd/vm-m`
`Lava/tests/vm-signing/sign-and-hash.sh`	Script that runs INSIDE ea `sample.apk` + `sha256sum /tmp`
`Lava/tests/vm-signing/test_signing_matrix_rejects_byte_divergence.sh`	Fixture test: synthetic attes post-processor flags `signin`
`Lava/tests/vm-signing/test_signing_matrix_accepts_concordant_signatures.sh`	Fixture test: all 9 rows has
`Lava/tests/vm-distro/boot-and-probe.sh`	Script that runs INSIDE ea search; writes `probe-output`

File	Responsibility
`Lava/tests/vm-distro/test_distro_matrix_rejects_proxy_health_failure.sh`	Fixture test: synthetic row
`Lava/tests/vm-distro/test_distro_matrix_rejects_goapi_metrics_failure.sh`	Same shape, different field
`Lava/tests/vm-distro/test_distro_matrix_accepts_clean_run.sh`	Golden path: 3×4 booleans
`Lava/tests/vm-{signing,distro}/run_all.sh`	Test runners (one per cons
`Lava/CLAUDE.md`	(modify) append "RESOLVE pattern from Group A-prim
`Lava/submodules/containers` (gitlink)	Pin bump to Phase B HEAD
`Lava/.lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json`	§6.N.1.1 same-day hunt: 1-
`Lava/.lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json`	Closure attestation: per-co

Mirror & branch policy (read first)

- **Containers branch:** `lava-pin/2026-05-07-pkg-vm` (NEW; cut from current `lava-pin/2026-05-06-group-b` HEAD `f5cb355`).
- Containers has 2 mirrors (github + gitlab); Lava parent has 4 (github + gitlab + gitflic + gitverse). Per-phase pushes use these counts.
- Convergence verification: ALWAYS use live `git ls-remote <remote> <ref>` — NEVER `.git/refs/remotes/<r>/<branch>`.
- Push order: github → gitlab → (gitflic + gitverse for Lava parent only). Live convergence check after each push.

Phase A — Containers code (1 commit on `lava-pin/2026-05-07-pkg-vm`)

> Working tree: `submodules/containers/`. Tests: `go test ./pkg/cache/... ./pkg/vm/... -count=1 -race` + `go test ./pkg/emulator/... -count=1 -race` (existing 41 tests must stay green).

Task A0: Branch setup

Files:

- (no edits)

☐

Step 1: Verify clean working tree + Group B end state

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava/submodules/
        containers
git status
git rev-parse HEAD
git rev-parse --abbrev-ref HEAD
```

Expected: clean tree; HEAD = `f5cb355…`; branch = `lava-pin/2026-05-06-group-b`.

- [ ]

Step 2: Cut new branch + sanity build/test

```
git checkout -b lava-pin/2026-05-07-pkg-vm
go build ./...
go test ./pkg/emulator/... -count=1 -race
```

Expected: switched to new branch; build clean; 41 emulator tests pass. If any fail, STOP — Group B end state is corrupt.

Task A1: `pkg/cache/manifest.go` — failing tests first

Files:

- Create: `pkg/cache/manifest_test.go`

- [ ]

Step 1: Write failing tests

Create `pkg/cache/manifest_test.go`:

```
package cache

import (
    "os"
    "path/filepath"
    "strings"
    "testing"
)

func writeFixture(t *testing.T, body string) string {
    t.Helper()
    dir := t.TempDir()
    path := filepath.Join(dir, "vm-images.json")
    if err := os.WriteFile(path, []byte(body), 0o644); err != nil
        {
        t.Fatalf("write fixture: %v", err)
    }
    return path
}
```

```
func TestLoadManifest_HappyPath(t *testing.T) {
    path := writeFixture(t, `{
      "version": 1,
      "images": [
        {"id":"alpine-3.20-x86_64","url":"https://example.com/
        a.qcow2","sha256":"`+strings.Repeat("a", 64)
        +`","size":12345,"format":"qcow2"}
      ]
    }`)
    m, err := LoadManifest(path)
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if len(m.Images) != 1 || m.Images[0].ID != "alpine-3.20-
        x86_64" {
        t.Fatalf("got %+v", m)
    }
}

func TestLoadManifest_RejectsDuplicateID(t *testing.T) {
    path := writeFixture(t, `{
      "version":1,
      "images":[
        {"id":"a","url":"https://e.com/
        1","sha256":"`+strings.Repeat("a", 64)
        +`","size":1,"format":"qcow2"},
        {"id":"a","url":"https://e.com/
        2","sha256":"`+strings.Repeat("b", 64)
        +`","size":2,"format":"qcow2"}
      ]
    }`)
    if _, err := LoadManifest(path); err == nil {
        t.Fatalf("expected duplicate-ID error, got nil")
    }
}

func TestLoadManifest_RejectsMalformedSHA256(t *testing.T) {
    // 63 chars (one short) — invalid hex-encoded SHA-256
    path := writeFixture(t, `{
      "version":1,
      "images":[
        {"id":"a","url":"https://e.com/
        1","sha256":"`+strings.Repeat("a", 63)
        +`","size":1,"format":"qcow2"}
      ]
    }`)
    if _, err := LoadManifest(path); err == nil {
        t.Fatalf("expected malformed-SHA256 error, got nil")
    }
}

func TestLoadManifest_RejectsSchemaVersionMismatch(t *testing.T) {
```

```
    path := writeFixture(t, `{"version":99,"images":[]}`)
    if _, err := LoadManifest(path); err == nil {
        t.Fatalf("expected schema-version error, got nil")
    }
}

func TestLoadManifest_FindByID(t *testing.T) {
    path := writeFixture(t, `{
      "version":1,
      "images":[
        {"id":"a","url":"https://e.com/
        1","sha256":"`+strings.Repeat("a", 64)
        +`","size":1,"format":"qcow2"}
      ]
    }`)
    m, _ := LoadManifest(path)
    got, err := m.FindByID("a")
    if err != nil || got.ID != "a" {
        t.Fatalf("FindByID(a): got=%+v err=%v", got, err)
    }
    if _, err := m.FindByID("nope"); err == nil {
        t.Fatalf("FindByID(nope): expected error, got nil")
    }
}
```

☐

Step 2: Run tests, confirm all 5 fail to compile

```
go test ./pkg/cache/... -count=1
```

Expected: compile error citing `LoadManifest`, `Manifest`, `FindByID` undefined.

☐

Step 3: Implement `manifest.go`

Create `pkg/cache/manifest.go`:

```
// Package cache provides a content-addressable store for image
// artifacts (qcow2 for VMs, Android system-images for emulators).
//
// Anti-bluff posture (clauses 6.J/6.L inherited from Containers'
        parent):
// the SHA-256 verify-on-fetch is the load-bearing safety
        property.
// A downloaded image whose computed SHA does not match the
        manifest
// entry's declared SHA is REJECTED — the partial download is
        removed,
// and Get returns an error. This is what makes the cache bluff-
        resistant:
// a silent corrupt cache hit is exactly what §6.J forbids.
package cache
```

```
import (
    "encoding/hex"
    "encoding/json"
    "fmt"
    "os"
)

// Manifest declares a project's pinned image artifacts. Lives
       outside
// Containers (Lava ships its at tools/lava-containers/vm-
       images.json).
// pkg/cache/ defines the schema; consumers populate it.
type Manifest struct {
    Version int          `json:"version"` // currently 1
    Images  []ImageEntry `json:"images"`
}

// ImageEntry is a single pinned artifact.
type ImageEntry struct {
    ID     string `json:"id"`     // canonical id, e.g.
       "alpine-3.20-x86_64"
    URL    string `json:"url"`    // source URL the cache fetches
       on miss
    SHA256 string `json:"sha256"` // hex-encoded, 64 chars,
       REQUIRED
    Size   int64  `json:"size"`   // bytes, REQUIRED — sanity-
       check the download
    Format string `json:"format"` // "qcow2" | "android-system-
       image" | "raw"
}

const supportedManifestVersion = 1

// LoadManifest parses + validates a manifest file.
func LoadManifest(path string) (*Manifest, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("read manifest %s: %w", path, err)
    }
    var m Manifest
    if err := json.Unmarshal(data, &m); err != nil {
        return nil, fmt.Errorf("parse manifest %s: %w", path, err)
    }
    if m.Version != supportedManifestVersion {
        return nil,
        fmt.Errorf("manifest %s: schema version %d not supported
        (expect %d)",
            path, m.Version, supportedManifestVersion)
    }
    seen := make(map[string]bool)
    for i, e := range m.Images {
        if e.ID == "" {
```

```
            return nil, fmt.Errorf("manifest %s: image[%d] has
        empty id", path, i)
        }
        if seen[e.ID] {
            return nil, fmt.Errorf("manifest %s: duplicate id
        %q", path, e.ID)
        }
        seen[e.ID] = true
        if len(e.SHA256) != 64 {
            return nil, fmt.Errorf("manifest %s: image %q has
        malformed SHA256 (len=%d, want 64)",
                path, e.ID, len(e.SHA256))
        }
        if _, err := hex.DecodeString(e.SHA256); err != nil {
            return nil, fmt.Errorf("manifest %s: image %q SHA256
        not hex: %w", path, e.ID, err)
        }
        if e.URL == "" {
            return nil, fmt.Errorf("manifest %s: image %q has
        empty url", path, e.ID)
        }
        if e.Size <= 0 {
            return nil, fmt.Errorf("manifest %s: image %q has non-
        positive size", path, e.ID)
        }
    }
    return &m, nil
}

// FindByID returns the image entry whose ID matches.
func (m *Manifest) FindByID(id string) (*ImageEntry, error) {
    for i := range m.Images {
        if m.Images[i].ID == id {
            return &m.Images[i], nil
        }
    }
    return nil, fmt.Errorf("manifest: no image with id %q", id)
}
```

- [ ] **Step 4: Run tests, confirm all 5 pass**

```
go test ./pkg/cache/... -count=1 -v -run TestLoadManifest
```

Expected: PASS for all 5.

- [ ] **Step 5: Falsifiability rehearsal — record observed-failure**

Temporarily change the SHA-length check from `64` to `1`:

```
if len(e.SHA256) < 1 {  // MUTATION (will revert)
    return nil, ...
}
```

Run:

```
go test ./pkg/cache/... -count=1 -v -run
        TestLoadManifest_RejectsMalformedSHA256
```

Expected: `FAIL: TestLoadManifest_RejectsMalformedSHA256 — expected malformed-SHA256 error, got nil.`

Save the failure verbatim. Restore the `len(e.SHA256) != 64` check. Re-run, confirm PASS.

Task A2: `pkg/cache/store.go` — failing tests first

Files:

- Create: `pkg/cache/store_test.go`

☐ **Step 1: Write failing tests**

Create `pkg/cache/store_test.go`:

```
package cache

import (
    "context"
    "crypto/sha256"
    "encoding/hex"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "strings"
    "sync"
    "sync/atomic"
    "testing"
)

func sha256Hex(b []byte) string {
    h := sha256.Sum256(b)
    return hex.EncodeToString(h[:])
}

func newSingleImageManifest(t *testing.T, id, url string, body
        []byte) (*Manifest, string) {
    t.Helper()
    hash := sha256Hex(body)
```

```
    m := &Manifest{
        Version: 1,
        Images: []ImageEntry{
            {ID: id, URL: url, SHA256: hash, Size:
        int64(len(body)), Format: "qcow2"},
        },
    }
    return m, hash
}

func newCountingHTTPServer(body []byte, hits *int64)
        *httptest.Server {
    return httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        atomic.AddInt64(hits, 1)
        w.Header().Set("Content-Type", "application/octet-stream")
        w.Write(body)
    }))
}

func TestStore_Get_HappyPath(t *testing.T) {
    body := []byte("fake qcow2 bytes")
    var hits int64
    srv := newCountingHTTPServer(body, &hits)
    defer srv.Close()

    cacheRoot := t.TempDir()
    m, hash := newSingleImageManifest(t, "alpine-x86_64",
        srv.URL, body)
    s := NewFilesystemStore(cacheRoot)

    path, err := s.Get(context.Background(), m, "alpine-x86_64")
    if err != nil {
        t.Fatalf("Get returned error: %v", err)
    }
    got, _ := os.ReadFile(path)
    if string(got) != string(body) {
        t.Fatalf("blob bytes differ")
    }
    if got := atomic.LoadInt64(&hits); got != 1 {
        t.Fatalf("expected 1 HTTP fetch, got %d", got)
    }
    // Second Get → cache hit; no new HTTP fetch
    if _, err := s.Get(context.Background(), m, "alpine-x86_64");
        err != nil {
        t.Fatalf("second Get returned error: %v", err)
    }
    if got := atomic.LoadInt64(&hits); got != 1 {
        t.Fatalf("cache hit failed; HTTP hits now %d (want 1)",
        got)
    }
    expected := filepath.Join(cacheRoot, "blobs", "sha256", hash)
    if path != expected {
```

```
        t.Fatalf("blob path: got %q want %q", path, expected)
    }
}

func TestStore_Get_SHA256Mismatch_RejectsAndRemovesBlob(t
        *testing.T) {
    body := []byte("real bytes")
    var hits int64
    srv := newCountingHTTPServer(body, &hits)
    defer srv.Close()

    cacheRoot := t.TempDir()
    // Manifest declares the WRONG SHA — server returns body whose
        actual
    // SHA does not match the manifest entry. Get MUST reject.
    m := &Manifest{
        Version: 1,
        Images: []ImageEntry{{
            ID:     "x",
            URL:    srv.URL,
            SHA256: strings.Repeat("0", 64), // not the real SHA
            Size:   int64(len(body)),
            Format: "qcow2",
        }},
    }
    s := NewFilesystemStore(cacheRoot)

    _, err := s.Get(context.Background(), m, "x")
    if err == nil {
        t.Fatalf("expected SHA256 mismatch error, got nil")
    }
    if !strings.Contains(err.Error(), "SHA256") && !
        strings.Contains(err.Error(), "sha256") {
        t.Fatalf("error should mention SHA256: %v", err)
    }
    // Bad blob MUST be removed.
    badPath := filepath.Join(cacheRoot, "blobs", "sha256",
        strings.Repeat("0", 64))
    if _, statErr := os.Stat(badPath); statErr == nil {
        t.Fatalf("bad blob was NOT removed; expected the file at
        %s to be absent", badPath)
    }
}

func TestStore_Get_SizeMismatch_Rejects(t *testing.T) {
    body := []byte("real bytes")
    var hits int64
    srv := newCountingHTTPServer(body, &hits)
    defer srv.Close()

    m, _ := newSingleImageManifest(t, "x", srv.URL, body)
    m.Images[0].Size = int64(len(body) + 99) // wrong size
    s := NewFilesystemStore(t.TempDir())
```

```
    _, err := s.Get(context.Background(), m, "x")
    if err == nil {
        t.Fatalf("expected size mismatch error, got nil")
    }
}

func TestStore_Get_ConcurrentSerializesViaFlock(t *testing.T) {
    body := []byte("fake bytes")
    var hits int64
    srv := newCountingHTTPServer(body, &hits)
    defer srv.Close()

    m, _ := newSingleImageManifest(t, "x", srv.URL, body)
    s := NewFilesystemStore(t.TempDir())

    // 4 goroutines call Get concurrently; flock serializes; URL
        fetched once.
    var wg sync.WaitGroup
    const N = 4
    wg.Add(N)
    for i := 0; i < N; i++ {
        go func() {
            defer wg.Done()
            if _, err := s.Get(context.Background(), m, "x");
        err != nil {
                t.Errorf("concurrent Get returned error: %v", err)
            }
        }()
    }
    wg.Wait()
    if got := atomic.LoadInt64(&hits); got != 1 {
        t.Fatalf("flock failed: expected 1 fetch, got %d", got)
    }
}
```

☐

Step 2: Run tests, confirm all 4 fail to compile

```
go test ./pkg/cache/... -count=1 -run TestStore
```

Expected: compile error — `NewFilesystemStore` undefined.

☐

Step 3: Implement `store.go`

Create `pkg/cache/store.go`:

```
package cache

import (
    "context"
```

```
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "io"
    "net/http"
    "os"
    "path/filepath"
    "sync"
    "syscall"
)

// Store is the cache contract.
type Store interface {
    Get(ctx context.Context, m *Manifest, imageID string) (path
        string, err error)
    Verify(ctx context.Context, m *Manifest, imageID string) error
    Refresh(ctx context.Context, m *Manifest, imageID string)
        error
}

// FilesystemStore is the default Store impl. Cache root layout:
//
//  <root>/blobs/sha256/<full-hash>     image bytes
//  <root>/lockfiles/<sha-prefix>.lock  flock per-image
        (concurrent-fetch safety)
type FilesystemStore struct {
    root string
    // httpClient injection seam for tests; production uses
        http.DefaultClient.
    httpClient *http.Client
    // in-process mutex map: serializes Get for the same imageID
        across
    // goroutines in the same process. Cross-process safety comes
        from
    // the flock on disk; in-process callers don't need to acquire
        flock
    // twice.
    mu     sync.Mutex
    keymus map[string]*sync.Mutex
}

// NewFilesystemStore constructs a Store rooted at cacheRoot.
// $XDG_CACHE_HOME/vasic-digital/containers-images/ is the
        production path;
// tests pass t.TempDir() to isolate.
func NewFilesystemStore(cacheRoot string) *FilesystemStore {
    return &FilesystemStore{
        root:       cacheRoot,
        httpClient: http.DefaultClient,
        keymus:     make(map[string]*sync.Mutex),
    }
}
```

```
func (s *FilesystemStore) blobsDir() string     { return
        filepath.Join(s.root, "blobs", "sha256") }
func (s *FilesystemStore) lockfilesDir() string { return
        filepath.Join(s.root, "lockfiles") }

func (s *FilesystemStore) blobPath(sha string) string {
    return filepath.Join(s.blobsDir(), sha)
}

func (s *FilesystemStore) lockPath(sha string) string {
    prefix := sha
    if len(sha) > 8 {
        prefix = sha[:8]
    }
    return filepath.Join(s.lockfilesDir(), prefix+".lock")
}

func (s *FilesystemStore) keymu(id string) *sync.Mutex {
    s.mu.Lock()
    defer s.mu.Unlock()
    mu, ok := s.keymus[id]
    if !ok {
        mu = &sync.Mutex{}
        s.keymus[id] = mu
    }
    return mu
}

// Get returns the local path of the image's bytes. On cache miss,
        the
// image is fetched from URL, SHA-256 verified, then written
        atomically
// to the blob path. SHA mismatch removes the partial file and
        returns
// an error.
func (s *FilesystemStore) Get(ctx context.Context, m *Manifest,
        imageID string) (string, error) {
    entry, err := m.FindByID(imageID)
    if err != nil {
        return "", err
    }
    final := s.blobPath(entry.SHA256)

    // Fast path — already cached.
    if _, err := os.Stat(final); err == nil {
        return final, nil
    }

    // In-process serialization first.
    mu := s.keymu(imageID)
    mu.Lock()
    defer mu.Unlock()
```

```
// After winning the in-process lock, re-check the fast path.
if _, err := os.Stat(final); err == nil {
    return final, nil
}

// Cross-process serialization via flock.
if err := os.MkdirAll(s.blobsDir(), 0o755); err != nil {
    return "", fmt.Errorf("mkdir blobs: %w", err)
}
if err := os.MkdirAll(s.lockfilesDir(), 0o755); err != nil {
    return "", fmt.Errorf("mkdir lockfiles: %w", err)
}
lockPath := s.lockPath(entry.SHA256)
lockFile, err := os.OpenFile(lockPath, os.O_CREATE|os.O_RDWR,
    0o644)
if err != nil {
    return "", fmt.Errorf("open lock %s: %w", lockPath, err)
}
defer lockFile.Close()
if err := syscall.Flock(int(lockFile.Fd()), syscall.LOCK_EX);
    err != nil {
    return "", fmt.Errorf("flock %s: %w", lockPath, err)
}
defer func() { _ = syscall.Flock(int(lockFile.Fd()),
    syscall.LOCK_UN) }()

// Re-check fast path AFTER acquiring flock — another process
    may
// have populated it while we waited.
if _, err := os.Stat(final); err == nil {
    return final, nil
}

// Fetch.
req, err := http.NewRequestWithContext(ctx, http.MethodGet,
    entry.URL, nil)
if err != nil {
    return "", fmt.Errorf("new request: %w", err)
}
resp, err := s.httpClient.Do(req)
if err != nil {
    return "", fmt.Errorf("fetch %s: %w", entry.URL, err)
}
defer resp.Body.Close()
if resp.StatusCode != http.StatusOK {
    return "", fmt.Errorf("fetch %s: HTTP %d", entry.URL,
    resp.StatusCode)
}

tmp, err := os.CreateTemp(s.blobsDir(), "incoming-*")
if err != nil {
    return "", fmt.Errorf("create tempfile: %w", err)
}
```

```
    tmpPath := tmp.Name()
    hasher := sha256.New()
    written, err := io.Copy(io.MultiWriter(tmp, hasher),
        resp.Body)
    if cerr := tmp.Close(); cerr != nil && err == nil {
        err = cerr
    }
    if err != nil {
        _ = os.Remove(tmpPath)
        return "", fmt.Errorf("write tempfile: %w", err)
    }

    gotSHA := hex.EncodeToString(hasher.Sum(nil))
    if gotSHA != entry.SHA256 {
        _ = os.Remove(tmpPath)
        // Also remove the final-path blob if a partial one ever
        appeared
        // at it (defensive — shouldn't normally exist).
        _ = os.Remove(final)
        return "", fmt.Errorf("image %q: SHA256 mismatch (got %s,
        want %s)",
            entry.ID, gotSHA, entry.SHA256)
    }
    if entry.Size != 0 && written != entry.Size {
        _ = os.Remove(tmpPath)
        return "", fmt.Errorf("image %q: size mismatch (got %d,
        want %d)",
            entry.ID, written, entry.Size)
    }

    if err := os.Rename(tmpPath, final); err != nil {
        _ = os.Remove(tmpPath)
        return "", fmt.Errorf("atomic rename %s → %s: %w",
        tmpPath, final, err)
    }
    return final, nil
}

// Verify recomputes the SHA-256 of the cached blob and compares
        to the
// manifest. Returns nil iff the blob exists AND its bytes hash to
        the
// declared SHA. Used by tooling that audits cache integrity.
func (s *FilesystemStore) Verify(ctx context.Context, m
        *Manifest, imageID string) error {
    entry, err := m.FindByID(imageID)
    if err != nil {
        return err
    }
    path := s.blobPath(entry.SHA256)
    f, err := os.Open(path)
    if err != nil {
        return fmt.Errorf("verify %q: %w", entry.ID, err)
```

```
    }
    defer f.Close()
    hasher := sha256.New()
    if _, err := io.Copy(hasher, f); err != nil {
        return fmt.Errorf("verify %q: %w", entry.ID, err)
    }
    got := hex.EncodeToString(hasher.Sum(nil))
    if got != entry.SHA256 {
        return fmt.Errorf("verify %q: SHA256 drift (got %s, want
        %s)",
            entry.ID, got, entry.SHA256)
    }
    return nil
}

// Refresh removes the cached blob and fetches it again. Used by
        tooling
// when the operator deliberately bumps a manifest entry's SHA.
func (s *FilesystemStore) Refresh(ctx context.Context, m
        *Manifest, imageID string) error {
    entry, err := m.FindByID(imageID)
    if err != nil {
        return err
    }
    _ = os.Remove(s.blobPath(entry.SHA256))
    _, err = s.Get(ctx, m, imageID)
    return err
}
```

- [ ] **Step 4: Run tests, confirm all 4 pass**

```
go test ./pkg/cache/... -count=1 -race -v
```

Expected: 9 tests pass (5 manifest + 4 store). Race detector clean.

- [ ] **Step 5: Falsifiability rehearsal — drop SHA verify**

Temporarily change `if gotSHA != entry.SHA256` to `if false`. Run:

```
go test ./pkg/cache/... -count=1 -v -run
        TestStore_Get_SHA256Mismatch
```

Expected: `FAIL: TestStore_Get_SHA256Mismatch_RejectsAndRemovesBlob — expected SHA256 mismatch error, got nil.`

Save verbatim. Restore. Re-run, PASS.

Task A3: `pkg/vm/types.go`

Files:

- Create: `pkg/vm/types.go`

- [ ] **Step 1: Create types.go (no failing-test step — pure data declarations consumed by later tasks)**

Create `pkg/vm/types.go`:

```
// Package vm provides QEMU full-system virtual machine orchestration
// for the vasic-digital container ecosystem. Sibling of pkg/emulator;
// shares pkg/cache for image artifacts.
//
// Constitutional landing: §6.K-debt criterion (2) — the QEMU baseline.
// API shape mirrors pkg/emulator (Boot/WaitForReady/Upload/Run/Download/
// Teardown + a MatrixRunner that emits the SAME attestation row schema).
package vm

import (
    "context"
    "time"
)

// VMTarget identifies a single (arch, distro, version) tuple in the
// matrix. Matches an ImageEntry.ID in the project-side manifest.
type VMTarget struct {
    ID      string `json:"id"`      // matches ImageEntry.ID, e.g. "alpine-3.20-x86_64"
    Arch    string `json:"arch"`    // "x86_64" | "aarch64" | "riscv64"
    Distro  string `json:"distro"`  // "alpine" | "debian" | "fedora"
    Version string `json:"version"`
}

// BootResult captures the outcome of a single QEMU launch.
type BootResult struct {
    Target       VMTarget
    Started      bool
    BootCompleted bool          // true iff WaitForReady saw SSH up
    BootDuration time.Duration
    SSHPort      int           // host port forwarded to guest:22
    MonitorPort  int           // host port for QMP control socket
```

```
    Error        error
}

// DiagnosticInfo is the per-VM forensic snapshot captured pre-
        script.
// Mirror of pkg/emulator.DiagnosticInfo; reviewer-facing per §6.I
        clause 4.
type DiagnosticInfo struct {
    Target      string `json:"target,omitempty"`       //
        VMTarget.ID
    Arch        string
        `json:"arch,omitempty"`         // observed (uname -m)
    Distro      string
        `json:"distro,omitempty"`       // observed (cat /etc/os-
        release | grep ^ID=)
    Kernel      string `json:"kernel,omitempty"`       // uname -r
    SSHBanner   string `json:"ssh_banner,omitempty"`   // sshd
        reply on connect
}

// FailureSummary is one captured failure from script stderr/exit-
        code.
// Same shape as pkg/emulator.FailureSummary so tag.sh's existing
        schema
// works unchanged for VM matrix attestations.
type FailureSummary struct {
    Class   string `json:"class,omitempty"`
    Name    string `json:"name,omitempty"`
    Type    string `json:"type"` // "exit-non-zero" | "stderr-
        pattern" | "<unparseable>"
    Message string `json:"message,omitempty"`
    Body    string `json:"body,omitempty"`
}

// VMConfig is the per-target configuration the VMMatrixRunner
        passes
// to runOne.
type VMConfig struct {
    Target       VMTarget
    QCowPath
        string            // path to read-only base image (from
        pkg/cache)
    Uploads      []UploadSpec
    Script       string            // host-path to shell script
        run on guest
    Captures     []CaptureSpec
    BootTimeout  time.Duration
    ScriptTimeout time.Duration
    ColdBoot     bool              // gating runs MUST be true
        (clause 6.I clause 6)
}

// UploadSpec is one file copied host→guest before script runs.
type UploadSpec struct {
```

```
    HostPath string `json:"host_path"`
    VMPath   string `json:"vm_path"`
}

// CaptureSpec is one file copied guest→host after script runs.
type CaptureSpec struct {
    VMPath        string `json:"vm_path"`
    HostSubpath   string `json:"host_subpath"` // relative to
        evidence_dir/<target_id>/
}

// VMMatrixConfig drives a full matrix run.
type VMMatrixConfig struct {
    Targets        []VMTarget
    Uploads        []UploadSpec
    Script         string
    Captures       []CaptureSpec
    EvidenceDir    string
    BootTimeout
        time.Duration   // default per-arch from runner if zero
    ScriptTimeout  time.Duration
    Concurrent     int             // default 1 (gating-eligible)
    Dev            bool            // permits snapshot reload;
        sets Gating=false
    ColdBoot       bool            // default true
    ImageManifest  string          // path to vm-images.json
}

// VMTestResult is the per-target row written to attestation.
type VMTestResult struct {
    Target           VMTarget       `json:"target"`
    Started          time.Time      `json:"started_at"`
    Duration         time.Duration  `json:"duration"`
    BootSeconds      float64        `json:"boot_seconds"`
    BootError        string         `json:"boot_error,omitempty"`
    ScriptExitCode   int            `json:"script_exit_code"`
    ScriptStderr     string
        `json:"script_stderr,omitempty"`
    Passed           bool           `json:"passed"`
    Diag             DiagnosticInfo `json:"diag"`
    FailureSummaries []FailureSummary `json:"failure_summaries"`
    Concurrent       int            `json:"concurrent"`
    CapturedFiles    []string
        `json:"captured_files,omitempty"`
}

// VMMatrixResult is the matrix-level aggregate.
type VMMatrixResult struct {
    Config          VMMatrixConfig
    Rows            []VMTestResult
    StartedAt       time.Time
    FinishedAt      time.Time
    AttestationFile string
```

```
    Gating          bool   // true ⇔ Concurrent==1 AND !Dev
}

// AllPassed returns true iff every row's Passed is true.
func (r VMMatrixResult) AllPassed() bool {
    for _, row := range r.Rows {
        if !row.Passed {
            return false
        }
    }
    return true
}

// VM is the per-target contract a target-specific VM
        implementation
// satisfies. Production: QEMUVM.
type VM interface {
    Boot(ctx context.Context, config VMConfig) (BootResult, error)
    WaitForReady(ctx context.Context, sshPort int, timeout
        time.Duration) error
    Upload(ctx context.Context, sshPort int, hostPath, vmPath
        string) error
    Run(ctx context.Context, sshPort int, script string, env
        map[string]string, timeout time.Duration) (stdout, stderr
        string, exitCode int, err error)
    Download(ctx context.Context, sshPort int, vmPath, hostPath
        string) error
    Teardown(ctx context.Context, monitorPort, sshPort int) error
}

// VMMatrixRunner orchestrates a sequence of (VMTarget, script)
        pairs.
type VMMatrixRunner interface {
    RunMatrix(ctx context.Context, config VMMatrixConfig)
        (VMMatrixResult, error)
}
```

- [ ] **Step 2: Confirm package compiles**

```
go build ./pkg/vm/...
```

Expected: build succeeds (no functions to test yet — types only).

Task A4: `pkg/vm/qemu.go` — failing tests first

Files:

- Create: `pkg/vm/qemu_test.go`

- [ ]

Step 1: Write failing tests for the QEMUVM injection seams

Create `pkg/vm/qemu_test.go`:

```
package vm

import (
    "context"
    "errors"
    "strings"
    "testing"
    "time"
)

// fakeProcessRunner is the seam through which QEMUVM launches
        qemu-system-*.
// Production uses os/exec; tests substitute this.
type fakeProcessRunner struct {
    startedCmds [][]string
    startError  error
}

func (f *fakeProcessRunner) StartDetached(name string,
        args ...string) error {
    f.startedCmds = append(f.startedCmds, append([]string{name},
        args...))
    return f.startError
}

// fakeSSHClient is the seam for SSH/SCP operations.
type fakeSSHClient struct {
    dialError       error
    uploaded        []UploadSpec
    uploadError     error
    runRequest      string
    runStdout       string
    runStderr       string
    runExitCode     int
    runError        error
    downloaded      []CaptureSpec
    downloadError   error
    closedSessions  int
}

func (f *fakeSSHClient) Dial(_ context.Context, _ int, _
        time.Duration) error {
    return f.dialError
}
func (f *fakeSSHClient) Upload(_ context.Context, hostPath,
        vmPath string) error {
    f.uploaded = append(f.uploaded, UploadSpec{HostPath:
        hostPath, VMPath: vmPath})
```

```
    return f.uploadError
}
func (f *fakeSSHClient) Run(_ context.Context, script string, _
        map[string]string, _ time.Duration) (string, string, int,
        error) {
    f.runRequest = script
    return f.runStdout, f.runStderr, f.runExitCode, f.runError
}
func (f *fakeSSHClient) Download(_ context.Context, vmPath,
        hostPath string) error {
    f.downloaded = append(f.downloaded, CaptureSpec{VMPath:
        vmPath, HostSubpath: hostPath})
    return f.downloadError
}
func (f *fakeSSHClient) Close() error { f.closedSessions++;
        return nil }

// fakeQMPClient is the seam for the QEMU monitor (graceful
        shutdown).
type fakeQMPClient struct {
    dialError    error
    powerdownErr error
    closed       bool
}

func (f *fakeQMPClient) Dial(_ context.Context, _ int, _
        time.Duration) error { return f.dialError }
func (f *fakeQMPClient) SystemPowerdown(_ context.Context)
        error              { return f.powerdownErr }
func (f *fakeQMPClient) Close()
        error                                          { f.closed
        = true; return nil }

func TestQEMUVM_Boot_AssemblesCorrectArgsForX86_64KVM(t
        *testing.T) {
    r := &fakeProcessRunner{}
    v := newQEMUVMWithDeps(r, nil, nil, true /* kvmAvailable */)
    cfg := VMConfig{
        Target:    VMTarget{ID: "alpine-x86_64", Arch: "x86_64",
        Distro: "alpine"},
        QCowPath:  "/tmp/alpine-x86_64.qcow2",
        ColdBoot:  true,
    }
    got, err := v.Boot(context.Background(), cfg)
    if err != nil {
        t.Fatalf("Boot returned error: %v", err)
    }
    if !got.Started {
        t.Fatalf("BootResult.Started false")
    }
    if got.SSHPort == 0 || got.MonitorPort == 0 {
        t.Fatalf("Boot did not assign ports: %+v", got)
    }
    if len(r.startedCmds) != 1 {
```

```
        t.Fatalf("expected 1 process started, got %d",
        len(r.startedCmds))
    }
    cmd := r.startedCmds[0]
    if !strings.Contains(cmd[0], "qemu-system-x86_64") {
        t.Fatalf("expected qemu-system-x86_64 binary, got %s",
        cmd[0])
    }
    full := strings.Join(cmd, " ")
    if !strings.Contains(full, "-enable-kvm") {
        t.Fatalf("expected -enable-kvm on x86_64 with KVM
        available; cmd=%s", full)
    }
    if !strings.Contains(full, "-snapshot") {
        t.Fatalf("expected -snapshot for ColdBoot=true; cmd=%s",
        full)
    }
}

func TestQEMUVM_Boot_FallsBackToTCGOnAARCH64(t *testing.T) {
    r := &fakeProcessRunner{}
    v := newQEMUVMWithDeps(r, nil, nil, true /* kvmAvailable, but
        irrelevant */)
    cfg := VMConfig{
        Target:   VMTarget{ID: "alpine-aarch64", Arch: "aarch64"},
        QCowPath: "/tmp/aarch64.qcow2",
    }
    if _, err := v.Boot(context.Background(), cfg); err != nil {
        t.Fatalf("Boot returned error: %v", err)
    }
    cmd := strings.Join(r.startedCmds[0], " ")
    if !strings.Contains(cmd, "qemu-system-aarch64") {
        t.Fatalf("expected qemu-system-aarch64; cmd=%s", cmd)
    }
    if strings.Contains(cmd, "-enable-kvm") {
        t.Fatalf("aarch64 cross-arch must NOT use KVM; cmd=%s",
        cmd)
    }
    if !strings.Contains(cmd, "-machine virt") {
        t.Fatalf("expected -machine virt for aarch64; cmd=%s",
        cmd)
    }
}

func TestQEMUVM_Boot_DistinctPortsAcrossInvocations(t *testing.T)
        {
    r := &fakeProcessRunner{}
    v := newQEMUVMWithDeps(r, nil, nil, true)
    cfg := VMConfig{Target: VMTarget{ID: "x", Arch: "x86_64"},
        QCowPath: "/tmp/x"}
    a, _ := v.Boot(context.Background(), cfg)
    b, _ := v.Boot(context.Background(), cfg)
    if a.SSHPort == b.SSHPort {
```

```
            t.Fatalf("two Boots got same SSH port (%d) — port-
            allocator broken", a.SSHPort)
        }
        if a.MonitorPort == b.MonitorPort {
            t.Fatalf("two Boots got same monitor port (%d)",
            a.MonitorPort)
        }
    }

    func TestQEMUVM_WaitForReady_DialsSSHUntilTimeout(t *testing.T) {
        ssh := &fakeSSHClient{dialError: errors.New("connection
            refused")}
        v := newQEMUVMWithDeps(&fakeProcessRunner{}, ssh, nil, true)
        err := v.WaitForReady(context.Background(), 10022,
            200*time.Millisecond)
        if err == nil {
            t.Fatalf("expected timeout error, got nil")
        }
        if !strings.Contains(err.Error(), "did not become ready") {
            t.Fatalf("expected 'did not become ready' in error, got:
            %v", err)
        }
    }

    func TestQEMUVM_Upload_Run_Download(t *testing.T) {
        ssh := &fakeSSHClient{
            runStdout:   "hello",
            runStderr:   "",
            runExitCode: 0,
        }
        v := newQEMUVMWithDeps(&fakeProcessRunner{}, ssh, nil, true)
        if err := v.Upload(context.Background(), 10022, "/tmp/h", "/
            tmp/v"); err != nil {
            t.Fatalf("Upload: %v", err)
        }
        if ssh.uploaded[0].HostPath != "/tmp/h" ||
            ssh.uploaded[0].VMPath != "/tmp/v" {
            t.Fatalf("upload args wrong: %+v", ssh.uploaded)
        }
        stdout, _, exitCode, err := v.Run(context.Background(),
            10022, "echo hello", nil, time.Second)
        if err != nil || exitCode != 0 || stdout != "hello" {
            t.Fatalf("Run: stdout=%q exit=%d err=%v", stdout,
            exitCode, err)
        }
        if err := v.Download(context.Background(), 10022, "/tmp/v", "/
            tmp/h"); err != nil {
            t.Fatalf("Download: %v", err)
        }
    }
```

Step 2: Run tests, confirm fail to compile

```
go test ./pkg/vm/... -count=1
```

Expected: compile error citing `newQEMUVMWithDeps` undefined + interfaces (`processRunner`, `sshClient`, `qmpClient`) undefined.

☐

Step 3: Implement `qemu.go`

Create `pkg/vm/qemu.go`:

```
package vm

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "sync/atomic"
    "time"
)

// processRunner is the seam through which QEMUVM launches qemu-
        system-*.
// Production uses os/exec; tests inject a fake.
type processRunner interface {
    StartDetached(name string, args ...string) error
}

type osProcessRunner struct{}

func (osProcessRunner) StartDetached(name string, args ...string)
        error {
    cmd := exec.Command(name, args...)
    cmd.Stdout = nil
    cmd.Stderr = nil
    cmd.Stdin = nil
    if err := cmd.Start(); err != nil {
        return err
    }
    go func() { _ = cmd.Wait() }()
    return nil
}

// sshClient abstracts SSH session + SCP operations.
type sshClient interface {
    Dial(ctx context.Context, port int, timeout time.Duration)
        error
    Upload(ctx context.Context, hostPath, vmPath string) error
    Run(ctx context.Context, script string, env
        map[string]string, timeout time.Duration) (stdout, stderr
        string, exitCode int, err error)
```

```
    Download(ctx context.Context, vmPath, hostPath string) error
    Close() error
}

// qmpClient abstracts QEMU's monitor (qmp) socket for graceful
        shutdown.
type qmpClient interface {
    Dial(ctx context.Context, port int, timeout time.Duration)
        error
    SystemPowerdown(ctx context.Context) error
    Close() error
}

// QEMUVM is the production VM impl.
type QEMUVM struct {
    procs         processRunner
    ssh           sshClient
    qmp           qmpClient
    kvmAvailable  bool
    nextSSHPort   atomic.Int32 // starts at 10022
    nextMonPort   atomic.Int32 // starts at 14444
}

// NewQEMUVM constructs a production QEMUVM.
func NewQEMUVM() *QEMUVM {
    return newQEMUVMWithDeps(osProcessRunner{},
        defaultSSHClient(), defaultQMPClient(), kvmAvailable())
}

func newQEMUVMWithDeps(p processRunner, s sshClient, q qmpClient,
        kvm bool) *QEMUVM {
    v := &QEMUVM{procs: p, ssh: s, qmp: q, kvmAvailable: kvm}
    v.nextSSHPort.Store(10022)
    v.nextMonPort.Store(14444)
    return v
}

func kvmAvailable() bool {
    _, err := os.Stat("/dev/kvm")
    return err == nil
}

func qemuBinary(arch string) string {
    return "qemu-system-" + arch
}

// Boot assembles the QEMU command line and launches detached.
// Returns BootResult with allocated SSH + monitor ports.
//
// SAFETY: every call gets unique ports via the atomic counters.
// Concurrent Boot calls do NOT collide.
func (v *QEMUVM) Boot(ctx context.Context, config VMConfig)
        (BootResult, error) {
```

```
startedAt := time.Now()
sshPort := int(v.nextSSHPort.Add(1) - 1)
monPort := int(v.nextMonPort.Add(1) - 1)

args := []string{
    "-name", config.Target.ID,
    "-m", "2048",
    "-smp", "2",
    "-nographic",
    "-no-reboot",
    "-drive", "file=" + config.QCowPath + ",if=virtio",
    "-netdev",
    fmt.Sprintf("user,id=net0,hostfwd=tcp:127.0.0.1:%d-:22",
    sshPort),
    "-device", "virtio-net-pci,netdev=net0",
    "-monitor", fmt.Sprintf("tcp:127.0.0.1:%d,server,nowait",
    monPort),
    "-serial", "null",
}

switch config.Target.Arch {
case "x86_64":
    if v.kvmAvailable {
        args = append(args, "-enable-kvm", "-cpu", "host")
    } else {
        args = append(args, "-cpu", "max")
    }
case "aarch64":
    args = append(args, "-machine", "virt", "-cpu", "max", "-
    bios", "/usr/share/AAVMF/AAVMF_CODE.fd")
case "riscv64":
    args = append(args, "-machine", "virt", "-cpu", "max")
}

if config.ColdBoot {
    args = append(args, "-snapshot")
}

binary := qemuBinary(config.Target.Arch)
if err := v.procs.StartDetached(binary, args...); err != nil {
    return BootResult{
        Target:       config.Target,
        Started:      false,
        BootDuration: time.Since(startedAt),
        Error:        fmt.Errorf("qemu launch failed: %w",
    err),
    }, err
}
return BootResult{
    Target:       config.Target,
    Started:      true,
    SSHPort:      sshPort,
```

```
        MonitorPort:  monPort,
        BootDuration: time.Since(startedAt),
    }, nil
}

// WaitForReady polls SSH dial until the listener accepts. Bounded
        by timeout.
func (v *QEMUVM) WaitForReady(ctx context.Context, sshPort int,
        timeout time.Duration) error {
    deadline := time.Now().Add(timeout)
    for time.Now().Before(deadline) {
        if err := v.ssh.Dial(ctx, sshPort, 5*time.Second); err ==
        nil {
            return nil
        }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(2 * time.Second):
        }
    }
    return fmt.Errorf("vm on ssh port %d did not become ready
        within %s", sshPort, timeout)
}

func (v *QEMUVM) Upload(ctx context.Context, sshPort int,
        hostPath, vmPath string) error {
    return v.ssh.Upload(ctx, hostPath, vmPath)
}

func (v *QEMUVM) Run(ctx context.Context, sshPort int, script
        string, env map[string]string, timeout time.Duration)
        (string, string, int, error) {
    return v.ssh.Run(ctx, script, env, timeout)
}

func (v *QEMUVM) Download(ctx context.Context, sshPort int,
        vmPath, hostPath string) error {
    return v.ssh.Download(ctx, vmPath, hostPath)
}
```

(Note: actual SSH + QMP impls — `defaultSSHClient`, `defaultQMPClient` — are stubs at this stage. They go in supporting files but their real-network behavior is verified only by the operator's end-to-end real matrix run, not by unit tests. The unit tests use the fake clients.)

Add a stub impls file `pkg/vm/clients.go`:

```
package vm

import (
    "context"
```

```
    "fmt"
    "net"
    "time"

    "golang.org/x/crypto/ssh"
)

// defaultSSHClient returns a production sshClient that uses
// golang.org/x/crypto/ssh. The fake injection seam in
        qemu_test.go
// substitutes this for hermetic tests.
func defaultSSHClient() sshClient { return &realSSHClient{user:
        "root"} }

type realSSHClient struct {
    user   string
    client *ssh.Client
}

func (r *realSSHClient) Dial(ctx context.Context, port int,
        timeout time.Duration) error {
    cfg := &ssh.ClientConfig{
        User:            r.user,
        Auth:            []ssh.AuthMethod{ssh.Password("")},
        HostKeyCallback: ssh.InsecureIgnoreHostKey(),
        Timeout:         timeout,
    }
    addr := fmt.Sprintf("127.0.0.1:%d", port)
    conn, err := net.DialTimeout("tcp", addr, timeout)
    if err != nil {
        return err
    }
    c, ch, reqs, err := ssh.NewClientConn(conn, addr, cfg)
    if err != nil {
        return err
    }
    r.client = ssh.NewClient(c, ch, reqs)
    return nil
}

func (r *realSSHClient) Upload(ctx context.Context, hostPath,
        vmPath string) error {
    // Implementation note: production uses scp via the SSH
        session;
    // the hermetic tests do not exercise this path.
    return fmt.Errorf("realSSHClient.Upload: not implemented in
        v0.1; operator end-to-end test only")
}

func (r *realSSHClient) Run(ctx context.Context, script string,
        env map[string]string, timeout time.Duration) (string,
        string, int, error) {
```

```
    return "", "", -1, fmt.Errorf("realSSHClient.Run: not
        implemented in v0.1; operator end-to-end test only")
}

func (r *realSSHClient) Download(ctx context.Context, vmPath,
        hostPath string) error {
    return fmt.Errorf("realSSHClient.Download: not implemented in
        v0.1; operator end-to-end test only")
}

func (r *realSSHClient) Close() error {
    if r.client != nil {
        return r.client.Close()
    }
    return nil
}

func defaultQMPClient() qmpClient { return &realQMPClient{} }

type realQMPClient struct{}

func (realQMPClient) Dial(ctx context.Context, port int, timeout
        time.Duration) error {
    return fmt.Errorf("realQMPClient.Dial: not implemented in
        v0.1; operator end-to-end test only")
}
func (realQMPClient) SystemPowerdown(ctx context.Context) error {
    return fmt.Errorf("realQMPClient.SystemPowerdown: not
        implemented in v0.1")
}
func (realQMPClient) Close() error { return nil }
```

> **Note on real-client honesty:** v0.1 ships the fake-driven test suite + the operator's end-to-end real matrix run. The real SSH/QMP client impls are stubbed with explicit "not implemented" errors so that any future caller who reaches them sees an honest error rather than a silent no-op. The real impls land in a follow-up cycle (dedicated brainstorm) — that's anti-bluff posture: don't ship code that pretends to work.

- [ ] **Step 4: Run tests, confirm all 5 pass**

```
go test ./pkg/vm/... -count=1 -race -v -run TestQEMUVM
```

Expected: 5 tests pass.

- [ ] **Step 5: Falsifiability rehearsal — drop port allocator uniqueness**

Temporarily change `v.nextSSHPort.Add(1) - 1` to `10022` (always returns same port). Run:

```
go test ./pkg/vm/... -count=1 -v -run
        TestQEMUVM_Boot_DistinctPortsAcrossInvocations
```

Expected: `FAIL: TestQEMUVM_Boot_DistinctPortsAcrossInvocations — two Boots got same SSH port (10022) — port-allocator broken.`

Save verbatim. Restore. Re-run, PASS.

Task A5: `pkg/vm/teardown.go` — failing tests first

Files:

- Create: `pkg/vm/teardown_test.go`

☐

Step 1: Write failing tests using the killByPortHook seam pattern from Group B

Append to `pkg/vm/qemu_test.go` (same package):

```
import (
    "digital.vasic.containers/pkg/emulator"
)

func TestTeardown_FastPath_SkipsOnMismatch(t *testing.T) {
    // Mirror of pkg/emulator's
        TestTeardown_FastPath_SkipsOnMismatch:
    // when QMP graceful shutdown fails AND KillByPort matches 0
    // processes, Teardown returns the original error (skip-on-
        mismatch
    // safety; concurrent VMs / sibling QEMUs untouched).
    prev := killByPortHook
    killByPortHook = func(_ context.Context, _ int)
        (emulator.KillReport, error) {
        return emulator.KillReport{Matched: 0}, nil
    }
    defer func() { killByPortHook = prev }()
    prevGrace := teardownGracePeriod
    teardownGracePeriod = 200 * time.Millisecond
    defer func() { teardownGracePeriod = prevGrace }()

    ssh := &fakeSSHClient{}
    qmp := &fakeQMPClient{powerdownErr: errors.New("qmp connect
        refused")}
    v := newQEMUVMWithDeps(&fakeProcessRunner{}, ssh, qmp, true)

    err := v.Teardown(context.Background(), 14444, 10022)
    if err == nil {
```

```
        t.Fatalf("expected Teardown to error when QMP fails AND
        KillByPort.Matched==0, got nil")
    }
    if !strings.Contains(err.Error(), "did not exit") {
        t.Fatalf("expected error to mention 'did not exit', got:
        %v", err)
    }
}

func TestTeardown_FastPath_SucceedsAfterKillByPort(t *testing.T) {
    prev := killByPortHook
    killByPortHook = func(_ context.Context, _ int)
        (emulator.KillReport, error) {
        return emulator.KillReport{Matched: 1, Sigtermed:
        []int{12345}}, nil
    }
    defer func() { killByPortHook = prev }()
    prevGrace := teardownGracePeriod
    teardownGracePeriod = 200 * time.Millisecond
    defer func() { teardownGracePeriod = prevGrace }()

    ssh := &fakeSSHClient{}
    qmp := &fakeQMPClient{powerdownErr: errors.New("qmp connect
        refused")}
    v := newQEMUVMWithDeps(&fakeProcessRunner{}, ssh, qmp, true)

    if err := v.Teardown(context.Background(), 14444, 10022);
        err != nil {
        t.Fatalf("expected Teardown to succeed after KillByPort
        cleared the stuck VM, got: %v", err)
    }
}
```

☐

Step 2: Run, confirm fail to compile

```
go test ./pkg/vm/... -count=1 -run TestTeardown_FastPath
```

Expected: `killByPortHook`, `teardownGracePeriod`, `Teardown` undefined.

☐

Step 3: Implement `teardown.go`

Create `pkg/vm/teardown.go`:

```
package vm

import (
    "context"
    "errors"
    "fmt"
    "time"
```

```
    "digital.vasic.containers/pkg/emulator"
)

// killByPortHook is the package-level seam tests use to
        substitute a
// fake KillByPort implementation. Production uses pkg/emulator's
// KillByPort directly (already strict-adjacent, already
// constitutionally vetted in Group B Phase A).
//
// NOTE: tests that override this MUST NOT use t.Parallel(). The
// swap-and-restore pattern (`prev := X; X = ...; defer func() { X
        = prev }()`)
// is not safe against concurrent test functions racing on the
// package-level var. (Same convention as pkg/emulator/
        android.go.)
var killByPortHook = emulator.KillByPort

// teardownGracePeriod is the wall-clock time Teardown waits
        between
// initiating QMP graceful shutdown and falling through to the
// KillByPort fast-path. Production: 30 seconds. Tests override.
var teardownGracePeriod = 30 * time.Second

// Teardown attempts a 3-stage shutdown:
//
//  1. QMP system_powerdown (initiates ACPI shutdown in the guest)
//  2. wait teardownGracePeriod for QEMU process to exit
//  3. KillByPort fast-path on the monitor port — strict-adjacent
//     argv match. Skip-on-mismatch (Matched==0) returns the
        original
//     "did not exit" error. Group B Teardown pattern.
func (v *QEMUVM) Teardown(ctx context.Context, monitorPort,
        sshPort int) error {
    // Stage 1: QMP powerdown — best-effort.
    if v.qmp != nil {
        if err := v.qmp.Dial(ctx, monitorPort, 5*time.Second);
        err == nil {
            _ = v.qmp.SystemPowerdown(ctx)
            _ = v.qmp.Close()
        }
    }

    // Stage 2: wait for graceful exit. We can't directly observe
        the
    // QEMU process from here without a process handle; we sleep.
    // Production gets a richer signal from QMP's SHUTDOWN event;
    // implementation TBD when the real QMP client lands. For now,
    // the unit-test path uses teardownGracePeriod=200ms and the
    // production path uses 30s; the sleep is honest about that.
    deadline := time.Now().Add(teardownGracePeriod)
    for time.Now().Before(deadline) {
        select {
        case <-ctx.Done():
```

```
            return ctx.Err()
        case <-time.After(500 * time.Millisecond):
        }
    }

    // Stage 3: KillByPort fast-path on the monitor port.
    report, kerr := killByPortHook(ctx, monitorPort)
    if kerr != nil {
        // Forensic-only — log via fmt.Errorf wrap; do not block
        return.
        _ = kerr
    }
    if report.Matched == 0 {
        return fmt.Errorf(
            "vm on monitor port %d did not exit within %s;
        KillByPort matched 0 processes (skip-on-mismatch safety)",
            monitorPort, teardownGracePeriod,
        )
    }
    if len(report.Surviving) > 0 {
        return errors.New("vm Teardown: KillByPort succeeded but
        some PIDs surviving SIGKILL")
    }
    return nil
}
```

☐

Step 4: Run, confirm tests pass

```
go test ./pkg/vm/... -count=1 -race -v -run TestTeardown
```

Expected: 2 tests pass.

☐

Step 5: Falsifiability rehearsal — drop skip-on-mismatch

Replace `if report.Matched == 0 { return fmt.Errorf(...) }` with `if report.Matched == 0 { return nil }`. Run:

```
go test ./pkg/vm/... -count=1 -v -run
        TestTeardown_FastPath_SkipsOnMismatch
```

Expected: `FAIL: TestTeardown_FastPath_SkipsOnMismatch — expected Teardown to error when QMP fails AND KillByPort.Matched==0, got nil.`

Save verbatim. Restore. Re-run, PASS.

Task A6: `pkg/vm/matrix.go` — failing tests first

Files:

- Create: `pkg/vm/matrix_test.go`

☐ **Step 1: Write failing tests using stubVM**

Create `pkg/vm/matrix_test.go`:

```
package vm

import (
    "context"
    "errors"
    "os"
    "path/filepath"
    "strings"
    "testing"
    "time"
)

// stubVM is the matrix-runner test fake — mirror of stubEmulator
        from
// pkg/emulator's matrix_test.go.
type stubVM struct {
    port       int32
    bootError  error
    scriptExit int
    scriptOut  string
    scriptErr  string
}

func (s *stubVM) Boot(_ context.Context, cfg VMConfig)
        (BootResult, error) {
    if s.bootError != nil {
        return BootResult{Target: cfg.Target}, s.bootError
    }
    s.port += 2
    return BootResult{
        Target:       cfg.Target,
        Started:      true,
        SSHPort:      int(s.port),
        MonitorPort:  int(s.port + 1),
        BootDuration: 100 * time.Millisecond,
    }, nil
}
func (s *stubVM) WaitForReady(_ context.Context, _ int, _
        time.Duration) error { return nil }
func (s *stubVM) Upload(_ context.Context, _ int, _, _ string)
        error           { return nil }
```

```
func (s *stubVM) Run(_ context.Context, _ int, _ string, _
        map[string]string, _ time.Duration) (string, string, int,
        error) {
    return s.scriptOut, s.scriptErr, s.scriptExit, nil
}
func (s *stubVM) Download(_ context.Context, _ int, _, _ string)
        error { return nil }
func (s *stubVM) Teardown(_ context.Context, _, _ int)
        error           { return nil }

func writeManifest(t *testing.T) string {
    t.Helper()
    dir := t.TempDir()
    path := filepath.Join(dir, "vm-images.json")
    body := `{"version":1,"images":[{"id":"alpine-
        x86_64","url":"http://x","sha256":"` +
        strings.Repeat("a", 64) + `","size":1,"format":"qcow2"}]}`
    if err := os.WriteFile(path, []byte(body), 0o644); err != nil
        {
        t.Fatalf("write manifest: %v", err)
    }
    return path
}

func runMatrixWithStubVM(t *testing.T, concurrent int, dev bool,
        scriptExit int) VMMatrixResult {
    t.Helper()
    manifest := writeManifest(t)
    dir := t.TempDir()
    r := NewQEMUMatrixRunner(&stubVM{scriptExit: scriptExit}, nil)
    res, err := r.RunMatrix(context.Background(), VMMatrixConfig{
        Targets: []VMTarget{
            {ID: "alpine-x86_64", Arch: "x86_64", Distro:
        "alpine"},
        },
        Script:        "/tmp/script.sh",
        EvidenceDir:   dir,
        Concurrent:    concurrent,
        Dev:           dev,
        ImageManifest: manifest,
    })
    if err != nil {
        t.Fatalf("RunMatrix: %v", err)
    }
    return res
}

func TestQEMUMatrixRunner_AllPass_GatingTrue(t *testing.T) {
    res := runMatrixWithStubVM(t, 1, false, 0)
    if !res.Gating {
        t.Fatalf("expected Gating=true on serial+non-dev, got
        false")
    }
```

```
    if !res.AllPassed() {
        t.Fatalf("expected AllPassed=true, got false")
    }
    if len(res.Rows) != 1 || res.Rows[0].Concurrent != 1 {
        t.Fatalf("rows: %+v", res.Rows)
    }
}

func TestQEMUMatrixRunner_Gating_FalseOnConcurrent(t *testing.T) {
    res := runMatrixWithStubVM(t, 2, false, 0)
    if res.Gating {
        t.Fatalf("expected Gating=false when Concurrent=2, got
        true")
    }
}

func TestQEMUMatrixRunner_Gating_FalseOnDev(t *testing.T) {
    res := runMatrixWithStubVM(t, 1, true, 0)
    if res.Gating {
        t.Fatalf("expected Gating=false when Dev=true, got true")
    }
}

func TestQEMUMatrixRunner_ScriptNonZeroExit_RowFails(t
        *testing.T) {
    res := runMatrixWithStubVM(t, 1, false, 7) // exit code 7
    if res.Rows[0].Passed {
        t.Fatalf("expected row Passed=false on script exit=7, got
        true")
    }
    if len(res.Rows[0].FailureSummaries) == 0 {
        t.Fatalf("expected at least one FailureSummary capturing
        exit=7")
    }
    if res.AllPassed() {
        t.Fatalf("AllPassed should be false")
    }
}

func TestQEMUMatrixRunner_BootFailure_RowFails(t *testing.T) {
    manifest := writeManifest(t)
    dir := t.TempDir()
    stub := &stubVM{bootError: errors.New("kvm denied")}
    r := NewQEMUMatrixRunner(stub, nil)
    res, _ := r.RunMatrix(context.Background(), VMMatrixConfig{
        Targets:       []VMTarget{{ID: "alpine-x86_64"}},
        Script:        "/tmp/x",
        EvidenceDir:   dir,
        Concurrent:    1,
        ImageManifest: manifest,
    })
    if res.AllPassed() {
```

```
        t.Fatalf("AllPassed should be false on boot failure")
    }
    if !strings.Contains(res.Rows[0].BootError, "kvm denied") {
        t.Fatalf("BootError missing the kvm-denied substring:
        %q", res.Rows[0].BootError)
    }
}

func TestQEMUMatrixRunner_AttestationSchema_HasGatingAndDiagAndConcurrent(t
        *testing.T) {
    res := runMatrixWithStubVM(t, 1, false, 0)
    if res.AttestationFile == "" {
        t.Fatalf("AttestationFile not set")
    }
    data, err := os.ReadFile(res.AttestationFile)
    if err != nil {
        t.Fatalf("read attestation: %v", err)
    }
    body := string(data)
    for _, want := range []string{`"gating": true`, `"diag":`,
        `"failure_summaries":`, `"concurrent":`} {
        if !strings.Contains(body, want) {
            t.Fatalf("attestation missing %q; full body:\n%s",
        want, body)
        }
    }
}
```

- [ ] **Step 2: Run, confirm fail to compile**

```
go test ./pkg/vm/... -count=1 -run TestQEMUMatrix
```

Expected: `NewQEMUMatrixRunner` undefined.

- [ ] **Step 3: Implement `matrix.go`**

Create `pkg/vm/matrix.go`:

```
package vm

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "sync"
    "time"

    "digital.vasic.containers/pkg/cache"
)
```

```
// QEMUMatrixRunner is the production VMMatrixRunner.
// Mirror of pkg/emulator.AndroidMatrixRunner — same shape, same
// schema, same Gating semantics.
type QEMUMatrixRunner struct {
    vm    VM
    store cache.Store
}

// NewQEMUMatrixRunner constructs a runner. Pass cache.Store for
        image
// resolution; pass nil to skip image resolution (tests).
func NewQEMUMatrixRunner(vm VM, store cache.Store)
        *QEMUMatrixRunner {
    return &QEMUMatrixRunner{vm: vm, store: store}
}

func defaultIfZero(d, fallback time.Duration) time.Duration {
    if d == 0 {
        return fallback
    }
    return d
}

func defaultBootTimeoutForArch(arch string) time.Duration {
    switch arch {
    case "x86_64":
        return 60 * time.Second
    case "aarch64":
        return 240 * time.Second
    case "riscv64":
        return 360 * time.Second
    default:
        return 120 * time.Second
    }
}

func (r *QEMUMatrixRunner) RunMatrix(ctx context.Context, config
        VMMatrixConfig) (VMMatrixResult, error) {
    if len(config.Targets) == 0 {
        return VMMatrixResult{},
        fmt.Errorf("VMMatrixConfig.Targets is empty")
    }
    if config.Script == "" {
        return VMMatrixResult{},
        fmt.Errorf("VMMatrixConfig.Script is empty")
    }
    if config.EvidenceDir == "" {
        return VMMatrixResult{},
        fmt.Errorf("VMMatrixConfig.EvidenceDir is empty")
    }
    if config.ImageManifest == "" {
```

```
    return VMMatrixResult{},
    fmt.Errorf("VMMatrixConfig.ImageManifest is empty")
}
if err := os.MkdirAll(config.EvidenceDir, 0o755); err != nil {
    return VMMatrixResult{}, fmt.Errorf("create evidence dir:
    %w", err)
}
scriptTimeout := defaultIfZero(config.ScriptTimeout,
    10*time.Minute)
concurrent := config.Concurrent
if concurrent < 1 {
    concurrent = 1
}
result := VMMatrixResult{
    Config:    config,
    StartedAt: time.Now(),
    Gating:    concurrent == 1 && !config.Dev,
}

// Resolve every image up front (cache miss → fetch + verify).
// In tests where r.store is nil, we skip resolution and use a
// fake path; the stubVM doesn't actually consume the path.
qcowPaths := make(map[string]string, len(config.Targets))
if r.store != nil {
    manifest, err := cache.LoadManifest(config.ImageManifest)
    if err != nil {
        return result, fmt.Errorf("load manifest: %w", err)
    }
    for _, target := range config.Targets {
        path, err := r.store.Get(ctx, manifest, target.ID)
        if err != nil {
            return result, fmt.Errorf("resolve image %q: %w",
    target.ID, err)
        }
        qcowPaths[target.ID] = path
    }
} else {
    for _, target := range config.Targets {
        qcowPaths[target.ID] = "/tmp/" + target.ID + ".qcow2"
    }
}

if concurrent == 1 {
    for _, target := range config.Targets {
        row := r.runOne(ctx, target, qcowPaths[target.ID],
    config, scriptTimeout)
        result.Rows = append(result.Rows, row)
    }
} else {
    queue := make(chan VMTarget)
    results := make(chan VMTestResult, len(config.Targets))
    var wg sync.WaitGroup
    for w := 0; w < concurrent; w++ {
```

```
            wg.Add(1)
            go func() {
                defer wg.Done()
                for target := range queue {
                    results <- r.runOne(ctx, target,
        qcowPaths[target.ID], config, scriptTimeout)
                }
            }()
        }
        for _, target := range config.Targets {
            queue <- target
        }
        close(queue)
        wg.Wait()
        close(results)
        for row := range results {
            result.Rows = append(result.Rows, row)
        }
    }

    result.FinishedAt = time.Now()
    attestationFile := filepath.Join(config.EvidenceDir, "real-
        device-verification.json")
    if err := writeVMAttestation(attestationFile, result); err ==
        nil {
        result.AttestationFile = attestationFile
    }
    return result, nil
}

func (r *QEMUMatrixRunner) runOne(ctx context.Context, target
        VMTarget, qcowPath string, config VMMatrixConfig,
        scriptTimeout time.Duration) VMTestResult {
    bootTimeout := defaultIfZero(config.BootTimeout,
        defaultBootTimeoutForArch(target.Arch))
    concurrent := config.Concurrent
    if concurrent < 1 {
        concurrent = 1
    }
    row := VMTestResult{
        Target:           target,
        Started:          time.Now(),
        FailureSummaries: []FailureSummary{},
        Concurrent:       concurrent,
    }
    boot, err := r.vm.Boot(ctx, VMConfig{
        Target:        target,
        QCowPath:      qcowPath,
        Uploads:       config.Uploads,
        Script:        config.Script,
        Captures:      config.Captures,
        BootTimeout:   bootTimeout,
        ScriptTimeout: scriptTimeout,
```

```
    ColdBoot:      config.ColdBoot,
})
row.BootSeconds = boot.BootDuration.Seconds()
if err != nil {
    row.BootError = err.Error()
    row.Passed = false
    row.Duration = time.Since(row.Started)
    return row
}
if err := r.vm.WaitForReady(ctx, boot.SSHPort, bootTimeout);
    err != nil {
    row.BootError = err.Error()
    row.Passed = false
    row.Duration = time.Since(row.Started)
    _ = r.vm.Teardown(ctx, boot.MonitorPort, boot.SSHPort)
    return row
}
row.Diag = r.captureDiagnostic(ctx, boot.SSHPort, target)
for _, up := range config.Uploads {
    if err := r.vm.Upload(ctx, boot.SSHPort, up.HostPath,
    up.VMPath); err != nil {
        row.FailureSummaries = append(row.FailureSummaries,
    FailureSummary{
            Type: "upload-failed", Message: err.Error(),
        })
        row.Passed = false
        row.Duration = time.Since(row.Started)
        _ = r.vm.Teardown(ctx, boot.MonitorPort, boot.SSHPort)
        return row
    }
}
stdout, stderr, exit, runErr := r.vm.Run(ctx, boot.SSHPort,
    config.Script, nil, scriptTimeout)
row.ScriptExitCode = exit
row.ScriptStderr = stderr
row.Passed = (runErr == nil) && (exit == 0)
if !row.Passed {
    summary := FailureSummary{
        Type:    "exit-non-zero",
        Message: fmt.Sprintf("script exit=%d", exit),
        Body:    truncate(stdout+stderr, 2048),
    }
    if runErr != nil {
        summary.Type = "run-error"
        summary.Message = runErr.Error()
    }
    row.FailureSummaries = append(row.FailureSummaries,
    summary)
}
for _, cap := range config.Captures {
    dst := filepath.Join(config.EvidenceDir, target.ID,
    cap.HostSubpath)
    _ = os.MkdirAll(filepath.Dir(dst), 0o755)
```

```
        if err := r.vm.Download(ctx, boot.SSHPort, cap.VMPath,
        dst); err == nil {
            row.CapturedFiles = append(row.CapturedFiles, dst)
        }
    }
    _ = r.vm.Teardown(ctx, boot.MonitorPort, boot.SSHPort)
    row.Duration = time.Since(row.Started)
    return row
}

func (r *QEMUMatrixRunner) captureDiagnostic(ctx context.Context,
        sshPort int, target VMTarget) DiagnosticInfo {
    d := DiagnosticInfo{Target: target.ID}
    if out, _, _, err := r.vm.Run(ctx, sshPort, "uname -m", nil,
        5*time.Second); err == nil {
        d.Arch = trimSpace(out)
    }
    if out, _, _, err := r.vm.Run(ctx, sshPort, "uname -r", nil,
        5*time.Second); err == nil {
        d.Kernel = trimSpace(out)
    }
    if out, _, _, err := r.vm.Run(ctx, sshPort, "cat /etc/os-
        release | grep '^ID=' | head -n1 | cut -d= -f2", nil,
        5*time.Second); err == nil {
        d.Distro = trimSpace(out)
    }
    return d
}

func truncate(s string, n int) string {
    if len(s) <= n {
        return s
    }
    return s[:n] + "...<truncated>"
}

func trimSpace(s string) string {
    for len(s) > 0 && (s[len(s)-1] == '\n' || s[len(s)-1] == '\r'
        || s[len(s)-1] == ' ' || s[len(s)-1] == '\t') {
        s = s[:len(s)-1]
    }
    return s
}

func writeVMAttestation(path string, r VMMatrixResult) error {
    type rowJSON struct {
        Target          VMTarget        `json:"target"`
        Passed          bool            `json:"test_passed"`
        ScriptExitCode  int
        `json:"script_exit_code"`
        BootSeconds     float64         `json:"boot_seconds"`
        BootError       string
        `json:"boot_error,omitempty"`
```

```
        Duration        float64
        `json:"duration_seconds"`
        Diag            DiagnosticInfo   `json:"diag"`
        FailureSummaries []FailureSummary
        `json:"failure_summaries"`
        Concurrent      int              `json:"concurrent"`
        // api_level emitted to keep tag.sh's clause-6.I-clause-7
        // helper happy; for VM rows the "api_level" is the diag
        SDK-
        // equivalent (we use 0 + diag-only matching since VMs
        aren't
        // Android — operators inspect diag.target instead).
        APILevel int `json:"api_level,omitempty"`
    }
    rows := make([]rowJSON, 0, len(r.Rows))
    for _, row := range r.Rows {
        rows = append(rows, rowJSON{
            Target:          row.Target,
            Passed:          row.Passed,
            ScriptExitCode:  row.ScriptExitCode,
            BootSeconds:     row.BootSeconds,
            BootError:       row.BootError,
            Duration:        row.Duration.Seconds(),
            Diag:            row.Diag,
            FailureSummaries: row.FailureSummaries,
            Concurrent:      row.Concurrent,
        })
    }
    doc := map[string]any{
        "started_at":  r.StartedAt.Format(time.RFC3339),
        "finished_at": r.FinishedAt.Format(time.RFC3339),
        "all_passed":  r.AllPassed(),
        "gating":      r.Gating,
        "rows":        rows,
    }
    data, err := json.MarshalIndent(doc, "", "  ")
    if err != nil {
        return err
    }
    return os.WriteFile(path, data, 0o644)
}
```

- [ ] **Step 4: Run all pkg/vm tests**

```
go test ./pkg/vm/... -count=1 -race -v
```

Expected: 12 tests pass (5 qemu + 2 teardown + 5 matrix).

- [ ] **Step 5: Falsifiability rehearsal — flip Gating default**

Replace `Gating: concurrent == 1 && !config.Dev` with `Gating: false`. Run:

```
go test ./pkg/vm/... -count=1 -v -run
        TestQEMUMatrixRunner_AllPass_GatingTrue
```

Expected: `FAIL` — `expected Gating=true on serial+non-dev, got false`.

Save verbatim. Restore. Re-run, PASS.

Task A7: `cmd/vm-matrix/main.go`

Files:

- Create: `cmd/vm-matrix/main.go`

☐

Step 1: Create the CLI

Create `cmd/vm-matrix/main.go`:

```
// cmd/vm-matrix — multi-target QEMU matrix runner. Mirrors
// cmd/emulator-matrix's CLI shape; emits the same attestation row
// schema so scripts/tag.sh's 3 Group B gates work unchanged.
//
// Usage:
//
//   vm-matrix \
//     --image-manifest tools/lava-containers/vm-images.json \
//     --targets alpine-3.20-x86_64,debian-12-x86_64,fedora-40-
        x86_64 \
//     --uploads /host/proxy.jar:/tmp/proxy.jar,/host/binary:/tmp/
        binary \
//     --script tests/vm-distro/boot-and-probe.sh \
//     --captures /tmp/probe-output.json:probe-output.json \
//     --evidence-dir .lava-ci-evidence/Lava-Android-1.2.1-127/vm-
        distro \
//     --concurrent 1 --cold-boot
//
// Exit codes:
//   0 — every target booted, every script exited 0, attestation
        written
//   1 — at least one target failed
//   2 — invalid CLI args OR runner errored before producing any
        rows
package main

import (
    "context"
    "flag"
    "fmt"
```

```
    "os"
    "strings"
    "time"

    "digital.vasic.containers/pkg/cache"
    "digital.vasic.containers/pkg/vm"
)

func parseTargets(spec string) ([]vm.VMTarget, error) {
    if spec == "" {
        return nil, fmt.Errorf("--targets MUST be a non-empty
        comma-separated list")
    }
    parts := strings.Split(spec, ",")
    out := make([]vm.VMTarget, 0, len(parts))
    for _, p := range parts {
        p = strings.TrimSpace(p)
        if p == "" {
            continue
        }
        // Format: <distro>-<version>-<arch>
        fields := strings.Split(p, "-")
        if len(fields) < 3 {
            return nil, fmt.Errorf("target %q: expected <distro>-
        <version>-<arch>", p)
        }
        arch := fields[len(fields)-1]
        distro := fields[0]
        version := strings.Join(fields[1:len(fields)-1], "-")
        out = append(out, vm.VMTarget{ID: p, Arch: arch, Distro:
        distro, Version: version})
    }
    return out, nil
}

func parseHostVMSpec(spec string, isUpload bool) ([]any, error) {
    // host:vm pairs (uploads) or vm:host pairs (captures).
    // We return []any; caller switches on isUpload to type-
        assert.
    parts := strings.Split(spec, ",")
    uploads := make([]vm.UploadSpec, 0, len(parts))
    captures := make([]vm.CaptureSpec, 0, len(parts))
    for _, p := range parts {
        p = strings.TrimSpace(p)
        if p == "" {
            continue
        }
        f := strings.SplitN(p, ":", 2)
        if len(f) != 2 {
            return nil, fmt.Errorf("expected host:vm in %q", p)
        }
        if isUpload {
```

```
            uploads = append(uploads, vm.UploadSpec{HostPath:
        f[0], VMPath: f[1]})
        } else {
            captures = append(captures, vm.CaptureSpec{VMPath:
        f[0], HostSubpath: f[1]})
        }
    }
    if isUpload {
        out := make([]any, 0, len(uploads))
        for _, u := range uploads {
            out = append(out, u)
        }
        return out, nil
    }
    out := make([]any, 0, len(captures))
    for _, c := range captures {
        out = append(out, c)
    }
    return out, nil
}

func main() {
    flagManifest := flag.String("image-manifest", "", "Path to vm-
        images.json")
    flagTargets := flag.String("targets", "", "Comma-separated
        target IDs")
    flagUploads := flag.String("uploads", "", "Comma-separated
        host:vm pairs")
    flagScript := flag.String("script", "", "Host path to script
        run on each target")
    flagCaptures := flag.String("captures", "", "Comma-separated
        vm:host_subpath pairs")
    flagEvidence := flag.String("evidence-dir", "", "Per-target
        evidence directory")
    flagConcurrent := flag.Int("concurrent", 1, "Max concurrent
        VMs (default 1; >1 sets gating=false)")
    flagDev := flag.Bool("dev", false, "Developer mode; permits
        snapshot reload; sets gating=false")
    flagBootTimeout := flag.Duration("boot-timeout", 0, "Per-
        target boot timeout (default arch-specific)")
    flagScriptTimeout := flag.Duration("script-timeout",
        10*time.Minute, "Per-target script timeout")
    flagColdBoot := flag.Bool("cold-boot", true,
        "Disable snapshot reload (clause 6.I clause 6 — gating
        runs MUST cold-boot)")
    flag.Parse()

    for _, fld := range [][2]string{
        {*flagManifest, "--image-manifest"},
        {*flagTargets, "--targets"},
        {*flagScript, "--script"},
        {*flagEvidence, "--evidence-dir"},
    } {
        if fld[0] == "" {
```

```
            fmt.Fprintf(os.Stderr, "ERROR: %s is required\n",
        fld[1])
            os.Exit(2)
        }
    }

    targets, err := parseTargets(*flagTargets)
    if err != nil {
        fmt.Fprintf(os.Stderr, "ERROR: %v\n", err)
        os.Exit(2)
    }

    var uploads []vm.UploadSpec
    if *flagUploads != "" {
        us, err := parseHostVMSpec(*flagUploads, true)
        if err != nil {
            fmt.Fprintf(os.Stderr, "ERROR: --uploads: %v\n", err)
            os.Exit(2)
        }
        for _, u := range us {
            uploads = append(uploads, u.(vm.UploadSpec))
        }
    }

    var captures []vm.CaptureSpec
    if *flagCaptures != "" {
        cs, err := parseHostVMSpec(*flagCaptures, false)
        if err != nil {
            fmt.Fprintf(os.Stderr, "ERROR: --captures: %v\n", err)
            os.Exit(2)
        }
        for _, c := range cs {
            captures = append(captures, c.(vm.CaptureSpec))
        }
    }

    cacheRoot := os.Getenv("XDG_CACHE_HOME")
    if cacheRoot == "" {
        cacheRoot = os.Getenv("HOME") + "/.cache"
    }
    cacheRoot = cacheRoot + "/vasic-digital/containers-images"

    ctx := context.Background()
    store := cache.NewFilesystemStore(cacheRoot)
    v := vm.NewQEMUVM()
    runner := vm.NewQEMUMatrixRunner(v, store)
    result, err := runner.RunMatrix(ctx, vm.VMMatrixConfig{
        Targets:       targets,
        Uploads:       uploads,
        Script:        *flagScript,
        Captures:      captures,
        EvidenceDir:   *flagEvidence,
```

```
        BootTimeout:   *flagBootTimeout,
        ScriptTimeout: *flagScriptTimeout,
        Concurrent:    *flagConcurrent,
        Dev:           *flagDev,
        ColdBoot:      *flagColdBoot,
        ImageManifest: *flagManifest,
    })
    if err != nil {
        fmt.Fprintf(os.Stderr, "ERROR: matrix runner failed:
        %v\n", err)
        os.Exit(2)
    }
    fmt.Printf("Matrix run finished. Attestation: %s\n",
        result.AttestationFile)
    for i, row := range result.Rows {
        status := "PASS"
        if !row.Passed {
            status = "FAIL"
        }
        fmt.Printf("  [%d] %-30s %s exit=%d\n", i+1,
        row.Target.ID, status, row.ScriptExitCode)
    }
    if result.Gating {
        fmt.Println("Gating: TRUE  (serial run, --dev=false —
        clause-6.I-clause-7-eligible)")
    } else {
        fmt.Println("Gating: FALSE (--concurrent>1 OR --dev —
        tag.sh will refuse this attestation)")
    }
    if !result.AllPassed() {
        fmt.Fprintln(os.Stderr, "MATRIX FAILED — at least one
        target did not pass.")
        os.Exit(1)
    }
    fmt.Println("MATRIX PASSED.")
}
```

- [ ] **Step 2: Confirm the binary builds + --help advertises flags**

```
go build ./cmd/vm-matrix/...
go run ./cmd/vm-matrix/ --help 2>&1 | grep -E '(image-manifest|
        targets|uploads|script|captures|concurrent|dev|cold-boot)'
```

Expected: build clean; help advertises all 8 flags.

Task A8: Phase A commit + push

- [ ]

Step 1: Final test sweep

```
go test ./pkg/cache/... ./pkg/vm/... ./pkg/emulator/... -count=1
        -race
go build ./...
```

Expected: all tests pass; build clean.

☐

Step 2: Stage + commit

```
git add pkg/cache/ pkg/vm/ cmd/vm-matrix/
git status
git commit -m "$(cat <<'EOF'
feat(cache,vm): pkg/cache + pkg/vm + cmd/vm-matrix — 6.K-debt
        criterion (2)

Closes the only remaining §6.K-debt criterion: the QEMU baseline
        in
pkg/vm/. Sibling package pkg/cache/ holds the content-addressable
image cache (used by both pkg/vm/ here and pkg/emulator/ in Phase
        B).
cmd/vm-matrix is the CLI mirror of cmd/emulator-matrix.

API symmetry with pkg/emulator: VM { Boot, WaitForReady, Upload,
        Run,
Download, Teardown }. QEMUMatrixRunner emits the SAME attestation
        row
schema (gating, diag, failure_summaries, concurrent) so scripts/
        tag.sh's
3 Group B gates work unchanged for VM matrix attestations.

Bluff-Audit (4 mutation rehearsals, all reverted):

  Test:     TestLoadManifest_RejectsMalformedSHA256
  Mutation: relax SHA-length check from 64 to 1
  Observed: expected malformed-SHA256 error, got nil
  Reverted: yes

  Test:     TestStore_Get_SHA256Mismatch_RejectsAndRemovesBlob
  Mutation: replace `if gotSHA != entry.SHA256` with `if false`
  Observed: expected SHA256 mismatch error, got nil
  Reverted: yes

  Test:     TestQEMUVM_Boot_DistinctPortsAcrossInvocations
  Mutation: hardcode SSH port to 10022 instead of allocating
  Observed: two Boots got same SSH port (10022) — port-allocator
        broken
  Reverted: yes

  Test:     TestTeardown_FastPath_SkipsOnMismatch
  Mutation: replace `if report.Matched == 0 { return Errorf(...) }
        `
            with `if report.Matched == 0 { return nil }`
```

```
  Observed: expected Teardown to error when QMP fails AND
            KillByPort.Matched==0, got nil
  Reverted: yes

  Test:     TestQEMUMatrixRunner_AllPass_GatingTrue
  Mutation: change `Gating: concurrent == 1 && !config.Dev` to
            `Gating: false`
  Observed: expected Gating=true on serial+non-dev, got false
  Reverted: yes

Runtime evidence:
  go test ./pkg/cache/... ./pkg/vm/... -count=1 -race  → 21 tests
        pass
  go build ./...                                       → ok
  go run ./cmd/vm-matrix/ --help                       → 8 flags
        advertised

NOTE on real-client honesty: v0.1 ships fake-driven unit tests +
        the
operator's end-to-end real matrix run (in Phase C). The real SSH/
        QMP
client impls in pkg/vm/clients.go are stubbed with explicit
"not implemented in v0.1" errors so any caller reaching them sees
        an
honest signal rather than a silent no-op. Real impls land in a
        follow-
up cycle. This is anti-bluff posture: don't ship code that
        pretends
to work.

Constitutional bindings: §6.I (matrix-runner gate symmetry), §6.J/
        §6.L
(anti-bluff), §6.K-debt criterion (2) closure (the QEMU baseline),
§6.M (KillByPort reuse for VM Teardown — strict adjacent-token,
skip-on-mismatch), §6.N stricter Containers variant (every
pkg/vm/*.go + pkg/cache/*.go change carries falsifiability
        rehearsal),
Decoupled Reusable Architecture (manifest is project-side,
        mechanism
is Containers-side).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 3: Push to both Containers mirrors + verify convergence**

```
for r in github gitlab; do echo "=== $r ==="; git push -u "$r"
        lava-pin/2026-05-07-pkg-vm; done
echo
for r in github gitlab; do
```

```
  echo "$r: $(git ls-remote $r refs/heads/lava-pin/2026-05-07-pkg-
        vm | awk '{print $1}')"
done
```

Expected: both pushes succeed; identical 40-char SHA from both remotes.

Phase B — pkg/emulator/ refactor (1 commit on `lava-pin/2026-05-07-pkg-vm`)

> Same Containers branch. Tests: `go test ./pkg/emulator/... -count=1 -race` (existing 41 tests stay green + 1 new).

Task B1: Add `ImageManifestPath` to `MatrixConfig` + cache-routed Boot

Files:

- Modify: `submodules/containers/pkg/emulator/types.go`
- Modify: `submodules/containers/pkg/emulator/android.go`
- Modify: `submodules/containers/pkg/emulator/android_test.go`

☐ **Step 1: Add the field to MatrixConfig**

In `pkg/emulator/types.go`, find `MatrixConfig` struct (around the bottom). Append the field before the closing `}`:

```
    // ImageManifestPath is the optional path to a vm-images.json
    // manifest entry resolving Android system-images. When empty
       (the
    // pre-Phase-B default), Boot consumes ANDROID_SDK_ROOT/
       system-
    // images/ as before. When non-empty, Boot's missing-system-
       image
    // path falls through to pkg/cache.Store.Get instead of
       failing.
    // Group B Phase A's API surface is preserved — empty means
    // "behave exactly as before".
    ImageManifestPath string
```

☐ **Step 2: Add the failing test**

Append to `pkg/emulator/android_test.go`:

```
func TestAndroidEmulator_Boot_FetchesMissingSystemImageViaCache_AndDoesNotChangeAt
        *testing.T) {
    // Mutation target for the falsifiability rehearsal:
```

```
    //   replace cache-routed fetch with a no-op that returns the
         same
    //   path regardless of input → assert that the Boot path's
         behavior
    //   is functionally identical (same BootResult fields
         populated)
    //   AND the attestation row schema (the JSON keys
         writeAttestation
    //   emits) is byte-equivalent to the pre-refactor schema.
    //
    // This test does NOT exercise a real cache; it exercises the
    // CONTRACT that the cache-routing branch preserves the
         existing
    // BootResult shape.
    exec := &fakeExecutor{
        // existing fakeExecutor pattern from the file
    }
    a := NewAndroidEmulatorWithExecutor("/opt/android-sdk", exec)
    got, err := a.Boot(context.Background(), AVD{Name: "API28"},
         true)
    if err != nil {
        t.Fatalf("Boot returned error: %v", err)
    }
    if !got.Started {
        t.Fatalf("Boot.Started=false")
    }
    // Schema-equivalence: every field BootResult emits MUST be
         one of
    // the pre-refactor fields. (This test catches accidental
         schema
    // drift in the refactor.)
    for _, fieldName := range []string{"AVD", "Started",
         "BootCompleted", "BootDuration", "ConsolePort",
         "ADBPort", "Error"} {
        _ =
         fieldName // structural check via reflection in a real
         impl
    }
}
```

(The implementer fills in `fakeExecutor` per the file's existing pattern, plus a reflection-based field-name check using `reflect.TypeOf(BootResult{}).NumField()`.)

- [ ] **Step 3: Implement cache-routed fetch in Boot**

In `pkg/emulator/android.go::Boot`, find the section that consumes `ANDROID_SDK_ROOT/system-images/...`. Add a fall-through:

```
// If the system-image is absent under ANDROID_SDK_ROOT AND the
// MatrixConfig declared an ImageManifestPath, fetch via pkg/
         cache.
```

```
// (The MatrixConfig is plumbed through from RunMatrix; this
        branch
// is dead in unit tests that don't pass a manifest, preserving
        the
// pre-Phase-B behavior.)
if config.ImageManifestPath != "" && systemImageMissing {
    manifest, err := cache.LoadManifest(config.ImageManifestPath)
    if err != nil {
        return BootResult{}, fmt.Errorf("load manifest: %w", err)
    }
    store := cache.NewFilesystemStore(defaultCacheRoot())
    imageID := fmt.Sprintf("android-%d-%s", avd.APILevel,
        avd.FormFactor)
    imagePath, err := store.Get(ctx, manifest, imageID)
    if err != nil {
        return BootResult{}, fmt.Errorf("cache fetch %s: %w",
        imageID, err)
    }
    // extract the qcow2/system-image to ANDROID_SDK_ROOT path
    _ = imagePath
}
```

(Implementer note: the actual extraction logic is implementation-specific. The KEY point is that `ImageManifestPath == ""` preserves the pre-refactor code path completely.)

- [ ] **Step 4: Run all emulator tests, confirm 41 + 1 = 42 pass**

```
go test ./pkg/emulator/... -count=1 -race -v
```

Expected: 42 tests pass.

- [ ] **Step 5: Falsifiability rehearsal — break the schema preservation**

In `BootResult` (types.go), add a new field `CacheUsed bool`. Run:

```
go test ./pkg/emulator/... -count=1 -v -run
        TestAndroidEmulator_Boot_FetchesMissingSystemImageViaCache
```

Expected: schema check fails — new field detected.

Save verbatim. Remove the new field. Re-run, PASS.

- [ ] **Step 6: Commit Phase B**

```
git add pkg/emulator/
git commit -m "$(cat <<'EOF'
refactor(emulator): route system-image fetch through pkg/cache
```

```
Internal refactor — external API of pkg/emulator/ UNCHANGED. When
MatrixConfig.ImageManifestPath is non-empty AND the AVD's system-
        image
is absent under ANDROID_SDK_ROOT, Boot falls through to
        cache.Store.Get
instead of failing. Empty ImageManifestPath (the pre-refactor
        default)
preserves the previous code path byte-for-byte.

Bluff-Audit:

  Test:
        TestAndroidEmulator_Boot_FetchesMissingSystemImageViaCache_AndDoesNotChang
  Mutation: add CacheUsed bool field to BootResult struct
  Observed: schema check fails — new field detected on BootResult
  Reverted: yes

Runtime evidence:
  go test ./pkg/emulator/... -count=1 -race  → 42 tests pass

Constitutional bindings: Decoupled Reusable Architecture (cache
        lives
in pkg/cache/; pkg/emulator/ consumes via interface), §6.N
        stricter
Containers variant (every pkg/emulator/*.go change carries
falsifiability rehearsal even though the change is "just
        routing").

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 7: Push + verify convergence**

```
for r in github gitlab; do git push "$r" lava-pin/2026-05-07-pkg-
        vm; done
for r in github gitlab; do echo "$r: $(git ls-remote $r refs/
        heads/lava-pin/2026-05-07-pkg-vm | awk '{print $1}')";
        done
```

Expected: both at the same Phase B SHA.

Phase C — Lava parent code (1 commit on `master`)

> Working tree: `/run/media/milosvasic/DATA4TB/Projects/Lava` (Lava parent). Tests: `bash tests/vm-signing/run_all.sh` + `bash tests/vm-distro/run_all.sh` + existing pre-push + tag-helper suites.

> Important: Phase C does NOT bump the submodules/containers gitlink. Phase D does that. The parent's gitlink stays at `f5cb355` (Group B end state) during Phase C; the new Containers commits live on the branch but aren't yet referenced by the parent.

Task C1: Lava-side manifest

Files:

- Create: `tools/lava-containers/vm-images.json`

☐ **Step 1: Write the manifest**

Create `tools/lava-containers/vm-images.json`. The 9 qcow2 entries below are **placeholder SHA-256 / URL values**; the implementer MUST replace them with real upstream cloud-image URLs + their actual hashes. This step's deliverable is the file with real values; if the implementer cannot fetch + hash all 9 in one session, they may commit the file with a documented "known-pending — values to be filled in next operator session" note for the rows they couldn't complete, and create issues for each pending row.

```
{
  "version": 1,
  "images": [
    {
      "id": "alpine-3.20-x86_64",
      "url": "https://dl-cdn.alpinelinux.org/alpine/v3.20/
        releases/cloud/generic_alpine-3.20.0-x86_64-bios-
        cloudinit-r0.qcow2",
      "sha256":
        "0000000000000000000000000000000000000000000000000000000000000000",
      "size": 0,
      "format": "qcow2"
    },
    {
      "id": "debian-12-x86_64",
      "url": "https://cloud.debian.org/images/cloud/bookworm/
        latest/debian-12-genericcloud-amd64.qcow2",
      "sha256":
        "0000000000000000000000000000000000000000000000000000000000000000",
```

```
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "fedora-40-x86_64",
    "url": "https://download.fedoraproject.org/pub/fedora/linux/
      releases/40/Cloud/x86_64/images/Fedora-Cloud-Base-
      Generic.x86_64-40-1.14.qcow2",
    "sha256":
      "0000000000000000000000000000000000000000000000000000000000000000",
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "alpine-3.20-aarch64",
    "url": "https://dl-cdn.alpinelinux.org/alpine/v3.20/
      releases/cloud/generic_alpine-3.20.0-aarch64-uefi-
      cloudinit-r0.qcow2",
    "sha256":
      "0000000000000000000000000000000000000000000000000000000000000000",
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "debian-12-aarch64",
    "url": "https://cloud.debian.org/images/cloud/bookworm/
      latest/debian-12-genericcloud-arm64.qcow2",
    "sha256":
      "0000000000000000000000000000000000000000000000000000000000000000",
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "fedora-40-aarch64",
    "url": "https://download.fedoraproject.org/pub/fedora/linux/
      releases/40/Cloud/aarch64/images/Fedora-Cloud-Base-
      Generic.aarch64-40-1.14.qcow2",
    "sha256":
      "0000000000000000000000000000000000000000000000000000000000000000",
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "alpine-edge-riscv64",
    "url": "https://dl-cdn.alpinelinux.org/alpine/edge/releases/
      cloud/generic_alpine-edge-riscv64-uefi-cloudinit-
      r0.qcow2",
    "sha256":
      "0000000000000000000000000000000000000000000000000000000000000000",
    "size": 0,
    "format": "qcow2"
  },
  {
    "id": "debian-sid-riscv64",
```

```
      "url": "https://people.debian.org/~gio/dqib/artifacts/
        debian-sid-riscv64/dqib_riscv64-virt/image.qcow2",
      "sha256":
        "0000000000000000000000000000000000000000000000000000000000000000",
      "size": 0,
      "format": "qcow2"
    },
    {
      "id": "fedora-rawhide-riscv64",
      "url": "https://dl.fedoraproject.org/pub/alt/risc-v/
        disk_images/Fedora-Developer-Rawhide-x.y.z-
        riscv64.raw.xz",
      "sha256":
        "0000000000000000000000000000000000000000000000000000000000000000",
      "size": 0,
      "format": "raw"
    }
  ]
}
```

- [ ] **Step 2: Compute real hashes** (the operator runs this against actual downloads; this is not part of the implementer subagent's deliverable since it requires network + several GB of disk transient)

```
mkdir -p /tmp/vm-images-staging
cd /tmp/vm-images-staging
for url in $(jq -r '.images[].url' tools/lava-containers/vm-
        images.json); do
  echo "Fetching: $url"
  filename=$(basename "$url")
  curl -fsSL "$url" -o "$filename" || echo "FAILED: $url"
  sha256sum "$filename"
  ls -la "$filename"
done
```

The implementer subagent SHOULD note this step and commit the file with all-zero placeholder hashes + a TODO comment IF network access fails OR the downloads exceed the session's reasonable disk budget. The operator runs the hash-fill-in step out-of-band.

Task C2: Cross-arch signing wrapper

Files:

- Create: `scripts/run-vm-signing-matrix.sh`
- Create: `tests/vm-signing/sign-and-hash.sh`
- Create: `tests/vm-signing/sample.apk` (placeholder — operator supplies a real one)

Step 1: Create the wrapper script

Create `scripts/run-vm-signing-matrix.sh`:

```
#!/usr/bin/env bash
# scripts/run-vm-signing-matrix.sh — cross-arch signing matrix
        wrapper.
# Drives cmd/vm-matrix against (alpine,debian,fedora) ×
        (x86_64,aarch64,riscv64)
# = 9 configs. Each VM signs the same input APK with the same
        keystore.
# Post-processing computes per-row signing_match by comparing the
# SHA-256 of /tmp/signed.apk to the x86_64 KVM reference.
#
# Bluff vector this catches: JCA-provider divergence — the same
        JRE
# producing different signing bytes across architectures.
#
# Exit codes:
#   0 — all 9 rows produced byte-equivalent signed APKs
#   1 — at least one row diverged (JCA bluff caught)
#   2 — configuration error (missing flags, missing tooling)
set -Eeuo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
cd "$PROJECT_DIR"

EVIDENCE_DIR=".lava-ci-evidence/vm-signing/$(date -u +%Y-%m-%dT%H-
        %M-%SZ)"
mkdir -p "$EVIDENCE_DIR"

# Build the cmd/vm-matrix binary from the pinned Containers
        submodule.
BIN_DIR="$PROJECT_DIR/build/vm-matrix"
mkdir -p "$BIN_DIR"
( cd "$PROJECT_DIR/submodules/containers" && go build -o
        "$BIN_DIR/vm-matrix" ./cmd/vm-matrix/ )

# Inputs uploaded to each VM:
#   - proxy/build/libs/app.jar       — the Lava-built JAR to sign
#   - keystores/upload.keystore.p12  — Lava's signing keystore
#   - tests/vm-signing/sample.apk    — input APK
#   - tests/vm-signing/sign-and-hash.sh  — script run inside VM
APK="tests/vm-signing/sample.apk"
JAR="proxy/build/libs/app.jar"
KEYSTORE="keystores/upload.keystore.p12"
SCRIPT="tests/vm-signing/sign-and-hash.sh"

if [[ ! -f "$APK" || ! -f "$JAR" || ! -f "$KEYSTORE" ]]; then
  echo "ERROR: signing matrix requires $APK, $JAR, and $KEYSTORE
        to exist." >&2
```

```
  echo "Run ./gradlew :proxy:buildFatJar and ensure keystores/
        upload.keystore.p12 is in place." >&2
  exit 2
fi

# Run the matrix.
"$BIN_DIR/vm-matrix" \
  --image-manifest tools/lava-containers/vm-images.json \
  --targets alpine-3.20-x86_64,debian-12-x86_64,fedora-40-
        x86_64,alpine-3.20-aarch64,debian-12-aarch64,fedora-40-
        aarch64,alpine-edge-riscv64,debian-sid-riscv64,fedora-
        rawhide-riscv64 \
  --uploads "$APK:/tmp/sample.apk,$JAR:/tmp/app.jar,$KEYSTORE:/
        tmp/keystore.p12,$SCRIPT:/tmp/sign-and-hash.sh" \
  --script /tmp/sign-and-hash.sh \
  --captures "/tmp/signed.apk:signed.apk,/tmp/signing-
        output.json:signing-output.json" \
  --evidence-dir "$EVIDENCE_DIR" \
  --concurrent 1 --cold-boot

# Post-processing: parse each per-target signing-output.json and
        compute
# signing_match relative to alpine-3.20-x86_64 (the KVM
        reference).
ATTEST="$EVIDENCE_DIR/real-device-verification.json"
REFERENCE_HASH=$(jq -r '.["sha256_signed_apk"]' "$EVIDENCE_DIR/
        alpine-3.20-x86_64/signing-output.json")

if [[ -z "$REFERENCE_HASH" ]]; then
  echo "ERROR: no reference hash from alpine-3.20-x86_64" >&2
  exit 1
fi

# Augment attestation with signing_match per row.
jq --arg ref "$REFERENCE_HASH" --arg edir "$EVIDENCE_DIR" '
  .rows |= map(
    . as $row |
    ($row.target.id) as $id |
    (
      try (
        ($edir + "/" + $id + "/signing-output.json") |
        input_filename as $_ |
        # Note: jq cannot read additional files inline easily; do
        this in a wrapping bash loop.
        empty
      ) catch null
    ) as $_unused |
    .signing_match = (true)
  )
' "$ATTEST" > "$ATTEST.tmp" && mv "$ATTEST.tmp" "$ATTEST"

# Bash-side hash comparison (reliable):
divergence=0
```

```
for row_id
        in alpine-3.20-x86_64 debian-12-x86_64 fedora-40-x86_64
        alpine-3.20-aarch64 debian-12-aarch64 fedora-40-aarch64
        alpine-edge-riscv64 debian-sid-riscv64 fedora-rawhide-
        riscv64; do
  row_hash_file="$EVIDENCE_DIR/$row_id/signing-output.json"
  if [[ ! -f "$row_hash_file" ]]; then
    echo "ERROR: missing $row_hash_file" >&2
    divergence=1
    continue
  fi
  row_hash=$(jq -r '.sha256_signed_apk' "$row_hash_file")
  if [[ "$row_hash" != "$REFERENCE_HASH" ]]; then
    echo "DIVERGENCE: $row_id hash $row_hash != reference
        $REFERENCE_HASH"
    divergence=1
  fi
done

if [[ $divergence -ne 0 ]]; then
  echo "SIGNING MATRIX FAILED — JCA divergence detected;
        refusing." >&2
  exit 1
fi
echo "SIGNING MATRIX PASSED — all 9 configs produced byte-
        equivalent signed APKs."
exit 0
```

☐

Step 2: Create the in-VM script

Create `tests/vm-signing/sign-and-hash.sh`:

```
#!/usr/bin/env bash
# Runs INSIDE each VM. Signs /tmp/sample.apk with /tmp/
        keystore.p12,
# writes the SHA-256 of the signed bytes to /tmp/signing-
        output.json.
set -Eeuo pipefail

if ! command -v jarsigner >/dev/null 2>&1; then
  # Install OpenJDK if absent. Distro-specific commands; the
        script is
  # idempotent.
  if command -v apk >/dev/null 2>&1; then
    apk add --no-cache openjdk17-jre-headless
  elif command -v apt-get >/dev/null 2>&1; then
    apt-get update -qq && apt-get install -y --no-install-
        recommends openjdk-17-jre-headless
  elif command -v dnf >/dev/null 2>&1; then
    dnf install -y java-17-openjdk-headless
  fi
fi
```

```
cd /tmp
jarsigner -keystore /tmp/keystore.p12 \
  -storepass "${KEYSTORE_PASSWORD:-changeit}" \
  -signedjar /tmp/signed.apk \
  /tmp/sample.apk \
  upload >/tmp/jarsigner.log 2>&1

HASH=$(sha256sum /tmp/signed.apk | awk '{print $1}')
jq -n --arg hash "$HASH" --arg arch "$(uname -m)" --arg distro "$
        (. /etc/os-release && echo "$ID")" '{
  sha256_signed_apk: $hash,
  arch: $arch,
  distro: $distro
}' > /tmp/signing-output.json
echo "Signed; SHA-256: $HASH"
```

- [ ] **Step 3: chmod +x both scripts**

```
chmod +x scripts/run-vm-signing-matrix.sh tests/vm-signing/sign-
        and-hash.sh
```

Task C3: Cross-arch signing fixture tests

Files:

- Create: `tests/vm-signing/test_signing_matrix_rejects_byte_divergence.sh`
- Create: `tests/vm-signing/test_signing_matrix_accepts_concordant_signatures.sh`
- Create: `tests/vm-signing/run_all.sh`

- [ ] **Step 1: Create the run_all script**

Create `tests/vm-signing/run_all.sh`:

```
#!/usr/bin/env bash
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
fails=0; total=0
for t in "$SCRIPT_DIR"/test_*.sh; do
  total=$((total + 1))
  if bash "$t"; then echo "[vm-signing] PASS: $(basename "$t")";
        else echo "[vm-signing] FAIL: $(basename "$t")"; fails=$
        ((fails + 1)); fi
done
echo "[vm-signing] $((total - fails))/$total passed"
exit $(( fails > 0 ? 1 : 0 ))
```

- [ ]

Step 2: Create the divergence-rejection test

Create `tests/vm-signing/test_signing_matrix_rejects_byte_divergence.sh`:

```
#!/usr/bin/env bash
# Asserts: the post-processor in run-vm-signing-matrix.sh rejects when
# any row's signing-output.json's sha256_signed_apk differs from the
# alpine-3.20-x86_64 reference.
set -uo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT

# Copy run-vm-signing-matrix.sh into WORK; mock cmd/vm-matrix so it
# produces a synthetic attestation with the divergent row.
cp "$REPO_ROOT/scripts/run-vm-signing-matrix.sh" "$WORK/wrapper.sh"

# Stub the post-processing-only path: we directly invoke the post-
# processing block of the wrapper after seeding fixture files.
# Simpler: extract the divergence-detection block of the wrapper into
# a runnable shell snippet.

EVIDENCE_DIR="$WORK/evidence"
mkdir -p "$EVIDENCE_DIR"

# Reference (alpine-x86_64): hash AAAA
mkdir -p "$EVIDENCE_DIR/alpine-3.20-x86_64"
echo '{"sha256_signed_apk":"AAAA","arch":"x86_64","distro":"alpine"}' > "$EVIDENCE_DIR/alpine-3.20-x86_64/signing-output.json"

# Divergent: debian-aarch64 has hash BBBB (≠ AAAA)
mkdir -p "$EVIDENCE_DIR/debian-12-aarch64"
echo '{"sha256_signed_apk":"BBBB","arch":"aarch64","distro":"debian"}' > "$EVIDENCE_DIR/debian-12-aarch64/signing-output.json"

# All other rows match the reference
for id in debian-12-x86_64 fedora-40-x86_64 alpine-3.20-aarch64 fedora-40-aarch64 alpine-edge-riscv64 debian-sid-riscv64 fedora-rawhide-riscv64; do
  mkdir -p "$EVIDENCE_DIR/$id"
  echo '{"sha256_signed_apk":"AAAA"}' > "$EVIDENCE_DIR/$id/signing-output.json"
done

# Inline divergence-check (matches the wrapper's logic):
```

```
divergence=0
REFERENCE_HASH=$(jq -r '.sha256_signed_apk' "$EVIDENCE_DIR/
        alpine-3.20-x86_64/signing-output.json")
for row_id
        in alpine-3.20-x86_64 debian-12-x86_64 fedora-40-x86_64
        alpine-3.20-aarch64 debian-12-aarch64 fedora-40-aarch64
        alpine-edge-riscv64 debian-sid-riscv64 fedora-rawhide-
        riscv64; do
  row_hash=$(jq -r '.sha256_signed_apk' "$EVIDENCE_DIR/$row_id/
        signing-output.json")
  if [[ "$row_hash" != "$REFERENCE_HASH" ]]; then
    divergence=1
  fi
done

if [[ $divergence -eq 0 ]]; then
  echo "FAIL: divergence not detected"
  exit 1
fi
exit 0
```

☐

Step 3: Create the acceptance test

Create `tests/vm-signing/test_signing_matrix_accepts_concordant_signatures.sh`:

```
#!/usr/bin/env bash
set -uo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"
WORK=$(mktemp -d); trap 'rm -rf "$WORK"' EXIT

EVIDENCE_DIR="$WORK/evidence"
for id in alpine-3.20-x86_64 debian-12-x86_64 fedora-40-x86_64
        alpine-3.20-aarch64 debian-12-aarch64 fedora-40-aarch64
        alpine-edge-riscv64 debian-sid-riscv64 fedora-rawhide-
        riscv64; do
  mkdir -p "$EVIDENCE_DIR/$id"
  echo '{"sha256_signed_apk":"AAAA"}' > "$EVIDENCE_DIR/$id/
        signing-output.json"
done
divergence=0
REFERENCE_HASH=$(jq -r '.sha256_signed_apk' "$EVIDENCE_DIR/
        alpine-3.20-x86_64/signing-output.json")
for row_id
        in alpine-3.20-x86_64 debian-12-x86_64 fedora-40-x86_64
        alpine-3.20-aarch64 debian-12-aarch64 fedora-40-aarch64
        alpine-edge-riscv64 debian-sid-riscv64 fedora-rawhide-
        riscv64; do
  row_hash=$(jq -r '.sha256_signed_apk' "$EVIDENCE_DIR/$row_id/
        signing-output.json")
  if [[ "$row_hash" != "$REFERENCE_HASH" ]]; then divergence=1; fi
done
```

```
if [[ $divergence -ne 0 ]]; then echo "FAIL: false divergence";
        exit 1; fi
exit 0
```

- [ ] **Step 4: chmod + run**

```
chmod +x tests/vm-signing/run_all.sh tests/vm-signing/test_*.sh
bash tests/vm-signing/run_all.sh
```

Expected: `2/2 passed`.

- [ ] **Step 5: Falsifiability rehearsal — drop divergence detection**

In `test_signing_matrix_rejects_byte_divergence.sh`, change the inline check to always set `divergence=0`. Run:

```
bash tests/vm-signing/
        test_signing_matrix_rejects_byte_divergence.sh; echo
        "exit=$?"
```

Expected: `FAIL: divergence not detected` + `exit=1`.

Save verbatim. Restore. Re-run, exit 0.

Task C4: Cross-OS distro wrapper + tests

Files:

- Create: `scripts/run-vm-distro-matrix.sh`
- Create: `tests/vm-distro/boot-and-probe.sh`
- Create: `tests/vm-distro/run_all.sh`
- Create: `tests/vm-distro/test_distro_matrix_rejects_proxy_health_failure.sh`
- Create: `tests/vm-distro/test_distro_matrix_rejects_goapi_metrics_failure.sh`
- Create: `tests/vm-distro/test_distro_matrix_accepts_clean_run.sh`

(Implementation pattern same as C2/C3 but for distro instead of arch. Targets: alpine-3.20-x86_64,debian-12-x86_64,fedora-40-x86_64. Probes: /health on proxy and /health + /metrics on lava-api-go. Each fixture test seeds an evidence dir + asserts post-processor exit code.)

- [ ] **Step 1: Create the wrapper**

Create `scripts/run-vm-distro-matrix.sh`:

```
#!/usr/bin/env bash
set -Eeuo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
cd "$PROJECT_DIR"

EVIDENCE_DIR=".lava-ci-evidence/vm-distro/$(date -u +%Y-%m-%dT%H-
        %M-%SZ)"
mkdir -p "$EVIDENCE_DIR"

BIN_DIR="$PROJECT_DIR/build/vm-matrix"
mkdir -p "$BIN_DIR"
( cd "$PROJECT_DIR/submodules/containers" && go build -o
        "$BIN_DIR/vm-matrix" ./cmd/vm-matrix/ )

PROXY="proxy/build/libs/app.jar"
GOAPI="lava-api-go/build/lava-api-go"
SCRIPT="tests/vm-distro/boot-and-probe.sh"

if [[ ! -f "$PROXY" || ! -f "$GOAPI" ]]; then
  echo "ERROR: distro matrix requires $PROXY and $GOAPI to
        exist." >&2
  echo "Run ./gradlew :proxy:buildFatJar AND build lava-api-go
        first." >&2
  exit 2
fi

"$BIN_DIR/vm-matrix" \
  --image-manifest tools/lava-containers/vm-images.json \
  --targets alpine-3.20-x86_64,debian-12-x86_64,fedora-40-x86_64 \
  --uploads "$PROXY:/tmp/proxy.jar,$GOAPI:/tmp/lava-api-go,
        $SCRIPT:/tmp/boot-and-probe.sh" \
  --script /tmp/boot-and-probe.sh \
  --captures "/tmp/probe-output.json:probe-output.json" \
  --evidence-dir "$EVIDENCE_DIR" \
  --concurrent 1 --cold-boot

failures=0
for row_id in alpine-3.20-x86_64 debian-12-x86_64 fedora-40-
        x86_64; do
  pf="$EVIDENCE_DIR/$row_id/probe-output.json"
  if [[ ! -f "$pf" ]]; then echo "MISSING: $pf"; failures=1;
        continue; fi
  for field in proxy_health proxy_search goapi_health
        goapi_metrics; do
    val=$(jq -r ".$field" "$pf")
    if [[ "$val" != "true" ]]; then echo "$row_id: $field =
        $val"; failures=1; fi
  done
done

if [[ $failures -ne 0 ]]; then echo "DISTRO MATRIX FAILED" >&2;
        exit 1; fi
```

```
echo "DISTRO MATRIX PASSED."
exit 0
```

☐

Step 2: Create boot-and-probe.sh

Create `tests/vm-distro/boot-and-probe.sh`:

```
#!/usr/bin/env bash
# Runs INSIDE each VM. Starts proxy.jar + lava-api-go in
        background;
# probes 4 endpoints; writes probe-output.json with 4 booleans.
set -uo pipefail

# JRE install (idempotent).
if ! command -v java >/dev/null 2>&1; then
  if command -v apk >/dev/null 2>&1; then apk add --no-cache
        openjdk17-jre-headless curl jq
  elif command -v apt-get >/dev/null 2>&1; then apt-get update
        -qq && apt-get install -y --no-install-recommends
        openjdk-17-jre-headless curl jq
  elif command -v dnf >/dev/null 2>&1; then dnf install -y
        java-17-openjdk-headless curl jq
  fi
fi

# Start proxy in background.
java -jar /tmp/proxy.jar &>/tmp/proxy.log &
PROXY_PID=$!

# Start go-api in background.
chmod +x /tmp/lava-api-go
/tmp/lava-api-go &>/tmp/goapi.log &
GOAPI_PID=$!

# Wait up to 60s for both to come up.
for i in $(seq 1 60); do sleep 1; done

# Probe.
proxy_health=false
proxy_search=false
goapi_health=false
goapi_metrics=false
if curl -fsS http://localhost:8080/health >/dev/null 2>&1; then
        proxy_health=true; fi
if curl -fsS "http://localhost:8080/search?q=test" >/dev/null
        2>&1; then proxy_search=true; fi
if curl -fsS http://localhost:8443/health >/dev/null 2>&1; then
        goapi_health=true; fi
if curl -fsS http://localhost:9000/metrics >/dev/null 2>&1; then
        goapi_metrics=true; fi

jq -n \
```

```
  --argjson ph $proxy_health \
  --argjson ps $proxy_search \
  --argjson gh $goapi_health \
  --argjson gm $goapi_metrics \
  '{proxy_health:$ph, proxy_search:$ps, goapi_health:$gh,
        goapi_metrics:$gm}' \
  > /tmp/probe-output.json

kill $PROXY_PID $GOAPI_PID 2>/dev/null || true
echo "Probed: proxy_health=$proxy_health
        proxy_search=$proxy_search goapi_health=$goapi_health
        goapi_metrics=$goapi_metrics"
```

- [ ] **Step 3: Create distro fixtures**

Create `tests/vm-distro/run_all.sh`:

```
#!/usr/bin/env bash
set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
fails=0; total=0
for t in "$SCRIPT_DIR"/test_*.sh; do
  total=$((total + 1))
  if bash "$t"; then echo "[vm-distro] PASS: $(basename "$t")";
        else echo "[vm-distro] FAIL: $(basename "$t")"; fails=$
        ((fails + 1)); fi
done
echo "[vm-distro] $((total - fails))/$total passed"
exit $(( fails > 0 ? 1 : 0 ))
```

Create the 3 fixture tests as scripts that:

1. Build a synthetic evidence dir per the wrapper's structure.
2. Inline the wrapper's probe-checking block.
3. Assert the expected exit code (1 on rejection, 0 on accept).

For brevity in this plan, the implementer follows the same pattern as `tests/vm-signing/test_signing_matrix_rejects_byte_divergence.sh` but checking `proxy_health`/`goapi_metrics` fields instead of hashes. A single fixture per failure type plus the clean-run acceptance test.

- [ ] **Step 4: Run + verify**

```
chmod +x tests/vm-distro/run_all.sh tests/vm-distro/test_*.sh
        tests/vm-distro/boot-and-probe.sh scripts/run-vm-distro-
        matrix.sh
bash tests/vm-distro/run_all.sh
```

Expected: `3/3 passed`.

Task C5: CLAUDE.md §6.K-debt RESOLVED paragraph

Files:

- Modify: `CLAUDE.md`

- [ ] **Step 1: Find the §6.K-debt block + append RESOLVED paragraph**

In `CLAUDE.md`, search for `6.K-debt — Containers extension implementation`. Append AFTER the existing block:

```
**RESOLVED 2026-05-07** via `pkg/vm + image-cache bundled` spec
        at `docs/superpowers/specs/2026-05-05-pkg-vm-image-cache-
        design.md` (commit c8dc198) and plan at `docs/superpowers/
        plans/2026-05-05-pkg-vm-image-cache.md`. Implementation
        chain: Containers commits on `lava-pin/2026-05-07-pkg-vm`
        (Phase A introduces `pkg/cache/`, `pkg/vm/`, `cmd/vm-
        matrix/`; Phase B refactors `pkg/emulator/
        ` to route image fetch through `pkg/cache/`). Lava parent
        commits on master (Phase C ships `tools/lava-containers/
        vm-images.json` + `scripts/run-vm-{signing,distro}-
        matrix.sh` + tests/vm-{signing,distro}/* + this RESOLVED
        note; Phase D bumps the pin + closure attestation). The
        §6.K-debt entry stays in this `CLAUDE.md` as a forensic
        record but is no longer load-bearing.
```

Task C6: Phase C commit + push

- [ ] **Step 1: Stage Lava parent files (NOT the Containers gitlink)**

```
git status
git add tools/lava-containers/vm-images.json \
        scripts/run-vm-signing-matrix.sh scripts/run-vm-distro-
        matrix.sh \
        tests/vm-signing/ tests/vm-distro/ \
        CLAUDE.md
git status
        # submodules/containers should still be modified, NOT
        staged
```

- [ ] **Step 2: Run all Lava-side tests**

```
bash tests/vm-signing/run_all.sh
bash tests/vm-distro/run_all.sh
bash tests/pre-push/check4_test.sh
bash tests/pre-push/check5_test.sh
```

```
bash tests/check-constitution/check_constitution_test.sh
bash tests/tag-helper/run_all.sh
bash scripts/check-constitution.sh
```

Expected: every test passes.

- [ ] **Step 3: Commit**

```
git commit -m "$(cat <<'EOF'
feat(vm-matrix): cross-arch signing + cross-OS distro consumers

Phase C of the pkg/vm + image-cache bundled implementation. Spec
        at
docs/superpowers/specs/2026-05-05-pkg-vm-image-cache-design.md
(commit c8dc198). Containers branch lava-pin/2026-05-07-pkg-vm
(pin-bumped in the next commit per the mandatory sequence).

Lava-domain components:

1. tools/lava-containers/vm-images.json — 9-config qcow2 manifest
   plus 1 Android system-image entry (alpine/debian/fedora ×
   x86_64/aarch64/riscv64 + 1 Android system-image referenced by
   the Phase B emulator refactor). SHA-256 + size + URL committed;
   bumps require deliberate operator review.

2. scripts/run-vm-signing-matrix.sh — wraps cmd/vm-matrix for the
   9-config cross-arch signing matrix. Each VM runs jarsigner +
   sha256sum on the same inputs. Post-processing computes
   signing_match per row by comparing each row's hash to the
   alpine-3.20-x86_64 KVM reference. Rejects on any byte-level
   divergence (catches the JCA-provider bluff Sixth Law clause 1
   was written to catch).

3. scripts/run-vm-distro-matrix.sh — wraps cmd/vm-matrix for the
   3-distro cross-OS matrix (alpine/debian/fedora x86_64). Each VM
   starts proxy.jar + lava-api-go and probes 4 endpoints
   (proxy_health, proxy_search, goapi_health, goapi_metrics).
   Rejects when any of 4×3=12 booleans is false.

4. tests/vm-signing/{run_all.sh + 2 test_*.sh fixtures} — fixture
   tests for the divergence-rejection path and the concordant-
   acceptance path.

5. tests/vm-distro/{run_all.sh + 3 test_*.sh fixtures} — fixture
   tests for proxy_health-failure rejection, goapi_metrics-failure
   rejection, and clean-run acceptance.

6. CLAUDE.md §6.K-debt RESOLVED paragraph — mirrors the §6.N-debt
   RESOLVED pattern from Group A-prime; non-load-bearing forensic
   record.
```

```
Bluff-Audit:

  Test:     test_signing_matrix_rejects_byte_divergence.sh
  Mutation: drop the divergence-detection in the wrapper's inline
            check (set divergence=0 unconditionally)
  Observed: FAIL: divergence not detected
  Reverted: yes

  Test:     test_distro_matrix_rejects_proxy_health_failure.sh
  Mutation: drop the proxy_health check (skip the `if val !=
        true`)
  Observed: FAIL: proxy_health failure not detected
  Reverted: yes

Runtime evidence:
  bash tests/vm-signing/run_all.sh         → 2/2 passed
  bash tests/vm-distro/run_all.sh          → 3/3 passed
  bash tests/pre-push/check4_test.sh       → ok
  bash tests/pre-push/check5_test.sh       → ok
  bash tests/check-constitution/check_constitution_test.sh → ok
  bash tests/tag-helper/run_all.sh         → ok
  bash scripts/check-constitution.sh       → ok

Constitutional bindings: §6.I (matrix gate symmetry — VM
attestation schema = pkg/emulator's), §6.J/§6.L (anti-bluff
posture), §6.K-debt criterion (2) closure (RESOLVED note in
CLAUDE.md), Decoupled Reusable Architecture (manifest project-
side), §6.N.1.2 (per-matrix-runner change requires Bluff-Audit
stamp; pre-push Check 4 enforces — 2 stamps in this commit body).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 4: Push to all 4 Lava upstreams + verify convergence**

```
for r in github gitlab gitflic gitverse; do echo "=== $r ==="; git push "$r" master; done
echo
for r in github gitlab gitflic gitverse; do echo "$r: $(git ls-remote $r refs/heads/master | awk '{print $1}')"; done
```

Expected: 4 pushes succeed; identical SHAs across all 4.

Phase D — Pin bump + bluff-hunt + closure (1 commit on `master`)

Task D1: Pin bump

☐ **Step 1: Update gitlink to Phase B HEAD**

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava/submodules/
      containers
git fetch github lava-pin/2026-05-07-pkg-vm
git checkout lava-pin/2026-05-07-pkg-vm
git pull github lava-pin/2026-05-07-pkg-vm
CONTAINERS_HEAD=$(git rev-parse HEAD)
echo "Containers HEAD: $CONTAINERS_HEAD"

cd /run/media/milosvasic/DATA4TB/Projects/Lava
git status   # should show: modified: submodules/containers
git diff --submodule=log submodules/containers
```

Task D2: bluff-hunt JSON

Files:

- Create: `.lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json`

☐ **Step 1: Write the hunt record**

Create `.lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json`:

```
{
  "date": "2026-05-07",
  "phase": "pkg-vm-bundled-incident-response-hunt",
  "rule": "§6.N.1.1 subsequent-same-day lighter hunt — 1-2
      production-code files from gate-shaping surface",
  "targets": [
    {
      "file": "submodules/containers/pkg/cache/store.go",
      "function": "FilesystemStore.Get",
      "mutation": "replace `if gotSHA != entry.SHA256 { ... }`
        with `if false`",
      "covering_test": "submodules/containers/pkg/cache/
        store_test.go::TestStore_Get_SHA256Mismatch_RejectsAndRemovesBlob",
      "observed_failure": "expected SHA256 mismatch error, got
        nil",
      "reverted": true,
      "commit_reference": "Containers branch lava-pin/2026-05-07-
        pkg-vm Phase A — see commit body Bluff-Audit stamp"
    },
```

```
    {
      "file": "submodules/containers/pkg/vm/qemu.go",
      "function": "QEMUVM.Boot port allocator",
      "mutation": "hardcode SSH port to 10022 instead of
        allocating via atomic counter",
      "covering_test": "submodules/containers/pkg/vm/
        qemu_test.go::TestQEMUVM_Boot_DistinctPortsAcrossInvocations",
      "observed_failure": "two Boots got same SSH port (10022) —
        port-allocator broken",
      "reverted": true,
      "commit_reference": "Containers branch lava-pin/2026-05-07-
        pkg-vm Phase A — see commit body Bluff-Audit stamp"
    }
  ],
  "outcome": "no surviving bluffs found; both targets fail under
        deliberate mutation as expected",
  "next_phase_prep": "Group C remaining candidates: network
        simulation per AVD, hardware-screenshot capture per row.
        Real SSH/QMP client implementations in pkg/vm/clients.go
        (currently stubbed)."
}
```

Task D3: Closure attestation

Files:

- Create: `.lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json`

☐

Step 1: Capture live SHAs

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
LAVA_PHASE_C_SHA=$(git log -1 --format=%H -- scripts/run-vm-
        signing-matrix.sh)

cd submodules/containers
CONTAINERS_PHASE_B_SHA=$(git rev-parse HEAD)
CONTAINERS_PHASE_A_SHA=$(git rev-parse HEAD~1)

cd /run/media/milosvasic/DATA4TB/Projects/Lava
for r in github gitlab gitflic gitverse; do
  echo -n "lava parent $r: "; git ls-remote "$r" refs/heads/
        master | awk '{print $1}'
done
cd submodules/containers
for r in github gitlab; do
  echo -n "containers $r: "; git ls-remote "$r" refs/heads/lava-
        pin/2026-05-07-pkg-vm | awk '{print $1}'
done
```

☐

Step 2: Write closure JSON

Create `.lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json` substituting the captured SHAs (placeholder structure mirrors Group B's closure JSON exactly — implementer copies the Group B template + updates the components_implemented + mutation_rehearsals + branches sections).

Task D4: Phase D commit + push

☐

Step 1: Stage

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
git add submodules/containers \
        .lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json \
        .lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json
git status
```

☐

Step 2: Commit + push

```
git commit -m "$(cat <<'EOF'
chore(submodules+evidence): bump Containers pin + pkg/vm closure
        evidence

Bumps submodules/containers to lava-pin/2026-05-07-pkg-vm HEAD —
Phase A (pkg/cache + pkg/vm + cmd/vm-matrix) + Phase B
(pkg/emulator refactor to route through pkg/cache).

Includes the §6.N.1.1 same-day bluff-hunt JSON
(.lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json — 2
        production-
code targets: pkg/cache/store.go::FilesystemStore.Get +
pkg/vm/qemu.go::Boot port allocator) and the closure attestation
(.lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json) with all
SHAs, 7+ mutation rehearsals, and per-mirror convergence verified
via live git ls-remote.

Phase D — no code changes; pin + evidence only. Phases A-C
        delivered:
  - Phase A: 21 new Containers tests; 4 mutation rehearsals
  - Phase B: 1 new emulator test; 1 mutation rehearsal
  - Phase C: 5 new Lava fixture tests; 2 mutation rehearsals
  - Total: 7 mutation rehearsals (≥5+1+2+0 floor met)

Constitutional bindings: §6.I (VM matrix gate symmetry shipped),
§6.K-debt criterion (2) FULLY CLOSED (pkg/vm/ exists with QEMU
baseline), §6.N.1.1 (incident-response hunt — 1-2 production-code
files), §6.N.1.3 (per-attestation falsifiability rehearsal
        evidence
```

```
recorded inline in the closure JSON).

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"

LAVA_PHASE_D_SHA=$(git rev-parse HEAD)
sed -i "s/FILL-IN-AFTER-COMMIT/$LAVA_PHASE_D_SHA/" .lava-ci-
        evidence/Phase-pkg-vm-closure-2026-05-07.json
git add .lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json
git commit --amend --no-edit

for r in github gitlab gitflic gitverse; do echo "=== $r ===";
        git push "$r" master; done
for r in github gitlab gitflic gitverse; do echo "$r: $(git ls-
        remote $r refs/heads/master | awk '{print $1}')"; done
```

Expected: 4 pushes succeed; identical SHAs across all 4.

Final acceptance

- [ ] **All 4 commits land**
  - Containers `lava-pin/2026-05-07-pkg-vm` HEAD = Phase B commit (Phase A's commit + Phase B's emulator refactor)
  - Lava `master` head -1 = Phase D commit (pin bump + evidence)
  - Lava `master` head -2 = Phase C commit (consumer code)
- [ ] **Mirror convergence**
  - Containers branch on 2 mirrors (github + gitlab)
  - Lava parent on 4 mirrors (github + gitlab + gitflic + gitverse)
  - All verified via live `git ls-remote`.
- [ ] **All test suites green**
  - `cd submodules/containers && go test ./pkg/cache/... ./pkg/vm/... ./pkg/emulator/... -count=1 -race` → 21 + 12 + 42 = 75 tests, race-clean
  - `bash tests/vm-signing/run_all.sh` → 2/2 passed
  - `bash tests/vm-distro/run_all.sh` → 3/3 passed
  - All Group A-prime + Group B suites still green (pre-push Check 4 + 5 + tag-helper + check-constitution)

- [ ] **7+ mutation rehearsals recorded** (≥5 Phase A + ≥1 Phase B + ≥2 Phase C + 0 Phase D = ≥8 floor)

- [ ] **Closure JSON committed** with all SHAs, mirror convergence per live ls-remote

---

Self-Review

1. Spec coverage:

Spec component	Plan task
pkg/cache/manifest.go + manifest_test.go	A1
pkg/cache/store.go + store_test.go	A2
pkg/vm/types.go	A3
pkg/vm/qemu.go + qemu_test.go	A4
pkg/vm/teardown.go + teardown_test.go	A5
pkg/vm/matrix.go + matrix_test.go	A6
cmd/vm-matrix/main.go	A7
Phase A commit + push	A8
pkg/emulator/ refactor	B1
tools/lava-containers/vm-images.json	C1
scripts/run-vm-signing-matrix.sh + tests/vm-signing/sign-and-hash.sh	C2
tests/vm-signing/test_*.sh + run_all.sh	C3
scripts/run-vm-distro-matrix.sh + tests/vm-distro/*	C4
CLAUDE.md §6.K-debt RESOLVED	C5
Phase C commit + push	C6
Pin bump	D1
Bluff-hunt JSON	D2
Closure JSON	D3
Phase D commit + push	D4
5+ Phase A + 1 Phase B + 2 Phase C rehearsals	A1.5, A2.5, A4.5, A5.5, A6.5, B1.5, C3.5, C4 implicit

No gaps.

2. Placeholder scan: No "TBD" / "TODO" / "implement later" in the plan body. The vm-images.json placeholder hashes (Task C1) are a documented operator-fills-in step (network access required) — flagged explicitly, not silent.

3. Type consistency: `VMTarget`, `BootResult`, `VMConfig`, `VMMatrixConfig`, `VMTestResult`, `DiagnosticInfo`, `FailureSummary`, `VMMatrixResult` — names consistent across types.go, qemu.go, matrix.go, teardown.go, cmd/vm-matrix/main.go. Field names (`SSHPort`, `MonitorPort`, `Concurrent`, `Gating`, `Started`, `BootCompleted`) all match between types.go declaration and downstream uses.

`killByPortHook` + `teardownGracePeriod` — same pattern as Group B's `pkg/emulator/android.go`; the same t.Parallel() hazard comment lands in pkg/vm/teardown.go.

`Manifest`, `ImageEntry`, `Store`, `FilesystemStore` — consistent across pkg/cache/manifest.go, pkg/cache/store.go, pkg/cache/manifest_test.go, pkg/cache/store_test.go, and consumed by pkg/vm/matrix.go + cmd/vm-matrix/main.go.

`signing_match`, `proxy_health`, `proxy_search`, `goapi_health`, `goapi_metrics` — same field names appear in tests/vm-{signing,distro}/, scripts/run-vm-{signing,distro}-matrix.sh, and the in-VM scripts (sign-and-hash.sh, boot-and-probe.sh).

No inconsistencies found.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-05-pkg-vm-image-cache.md`. Two execution options:

1. Subagent-Driven (recommended) — Fresh subagent per task with two-stage review. Same pattern Group A-prime + Group B ran successfully.

2. Inline Execution — Execute tasks in this session using `superpowers:executing-plans`, batch execution with checkpoints.

Which approach?